DATA STRUCTURES AND ALGORITHMS

23CSE203

Submitted by	
Name	KANDULA GOWTHAM KOUSHIK
Roll No	AV.SC.U4CSE25119
Year/sem/section	2nd year/Sem-3/CSE-B
Credits	5

Submitted to	
Name	DR.RAJKUMAR BATCHU
Department	School of computing
Designation	Asst. professor(SI. Grade)

Index

Sno	Title/Topics	Page No.	Date
1.	**WEEK 1:** **Recursion** • **Recursive program for countdown from n to 1, followed by 'launch'.** • **Recursive program to calculate P^n (compound interest growth factor P, years n).** • **Recursive program to print first n terms of Fibonacci series.** • **Recursive program to search for an employee ID in a list.** • **Recursive program to calculate factorial (n!) for parcel arrangement.**	**5-12**	**14-07-26**
2.	**WEEK 2:** **Searching** • **Python program to perform Linear Search on given input.** • **Python program to perform Binary Search on given input (verify with sorted and unsorted lists).**	**13-18**	**21-07-26**
3.	**WEEK 3:** **Elementary Sorting** • **Python program to implement Bubble Sort (user input).** • **Python program to implement Selection Sort (user input).**	**19-23**	**01-08-26**

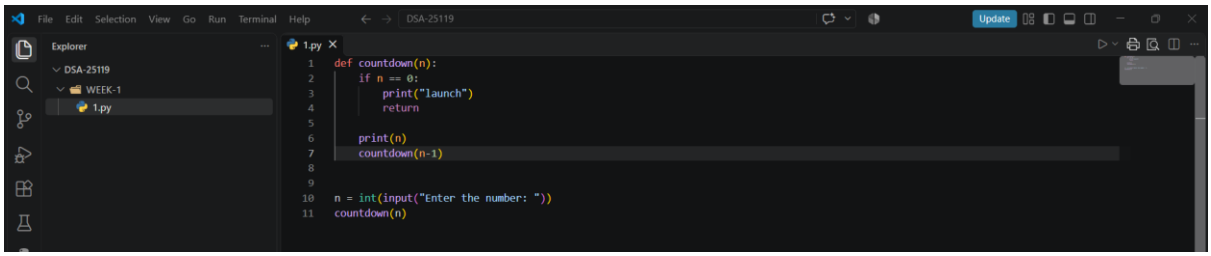
	• **Python program to implement Insertion Sort (user input).**		
4.	**WEEK 4:** **Advanced Sorting** • **Python program to implement Merge Sort (user input).** • **Python program to implement Quick Sort (user input).**	**24-30**	**11-08-26**
5.	**WEEK 5:** **Linked Lists** **Implement operations for:** • **Singly Linked List (SLL)** • **Doubly Linked List (DLL)** • **Circular Linked List (CLL)** **Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5.** **Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9.** **Display/Traverse 10. Exit**	**31-44**	**18-08-26**
6.	**WEEK 6:** **Stacks** • **Implement Stack operations (Push, Pop, Peek, Display) using Array.** • **Implement Stack operations (Push, Pop, Peek, Display) using Linked List.**	**45-50**	**25-08-26**

7.	**WEEK 7:** **Queues** **Implement Queue using Arrays.** **Implement Queue using Linked List.** **Implement Circular Queue using Arrays.** **Implement Circular Queue using Linked List.**	**51-58**	**22-09-26**
8.	**WEEK 8:** **Binary Trees** • **Implement Binary Tree with operations using Array.** • **Implement Binary Tree with operations using Linked List.**	**59-64**	**29-09-26**
9.	**WEEK 9:** **Binary Search Trees** **Python program to implement Binary Search Tree (BST) operations.**	**65-68**	**06-10-26**
10.	**WEEK 10:** **AVL Tree** **Write a Python program to implement an AVL Tree with all its operations.**	**69-73**	**13-10-26**
11.	**WEEK 11:** **Hashing** **Python program to implement Hashing (Separate chaining, Linear probing, and** **Quadratic probing).**	**74-76**	**27-10-26**

WEEK-1

1A. AIM: A Rocket launch system displays a countdown before launch. Instead of suing a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by “launch”.

Code:

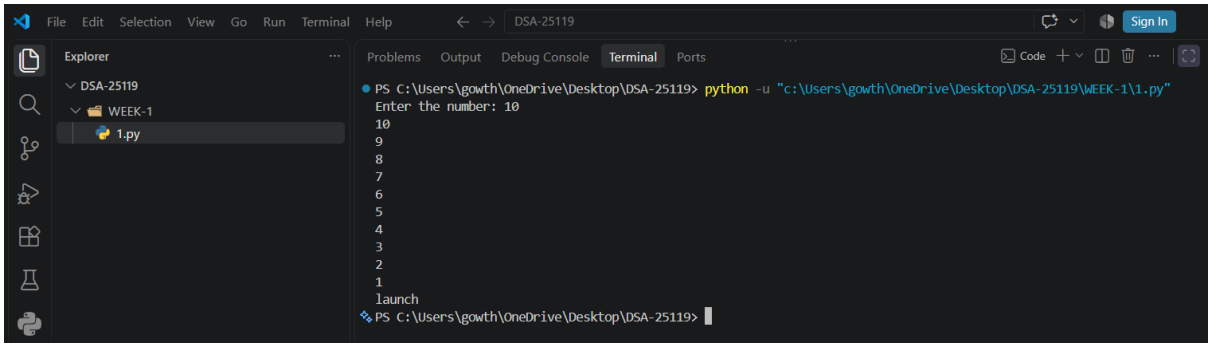


```python
def countdown(n):
    if n == 0:
        print("launch")
        return

    print(n)
    countdown(n-1)


n = int(input("Enter the number: "))
countdown(n)
```

Output:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-1\1.py"
Enter the number: 10
10
9
8
7
6
5
4
3
2
1
launch
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

Explanation :

Condition 1: Positive Input (n > 0)

- The function checks whether n == 0.
- Since n is positive, the condition is **False**.
- It executes:
  - print(n) → prints the current value of n.
  - countdown(n-1) → calls the same function again with n-1.
- This process continues until n becomes 0.
- When n == 0, the condition becomes **True**.
- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

Condition 2: Zero Input (n == 0)

- The function checks if n == 0.
- The condition is immediately **True**.
- It prints **launch**.
- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

Condition 3: Negative Input (n < 0)

- The condition n == 0 is **False**.
- It prints the negative value.
- Then it calls itself with n-1, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.
- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

Error table:

s.no	Error obtained	Rectification
1	Negative input in countdown	Add a condition to handle negative numbers before recursion.

1B. Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p(n) where p=principal growth, n= no. of years.

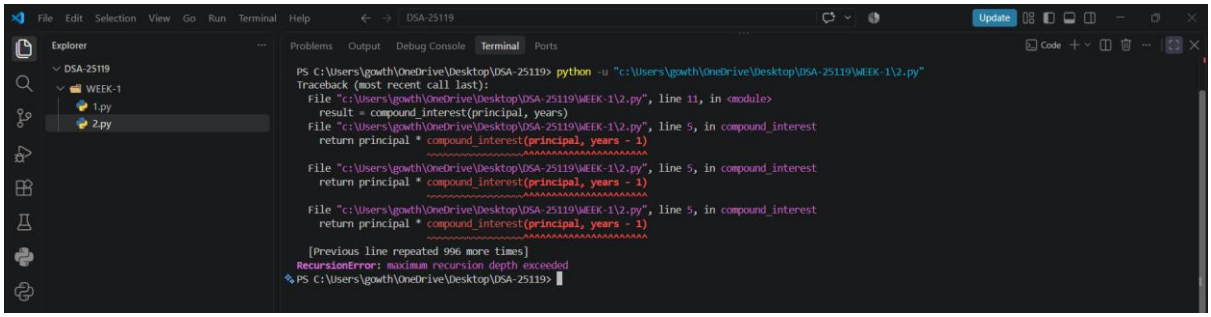
Code:



Output:



Negative output:



. The error is RecursionError: maximum recursion depth exceeded.

Explanation :

Condition 1: n > 0

- The function compound_interest(p, n) is called with the principal amount p and number of years n.
- The function checks whether n == 0.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:
  - p * compound_interest(p, n-1)
- The function calls itself recursively with n-1.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.
- Therefore, the function calculates $\mathbf{p^n}$.

Condition 2: n == 0

- When n becomes 0, the condition n == 0 becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.

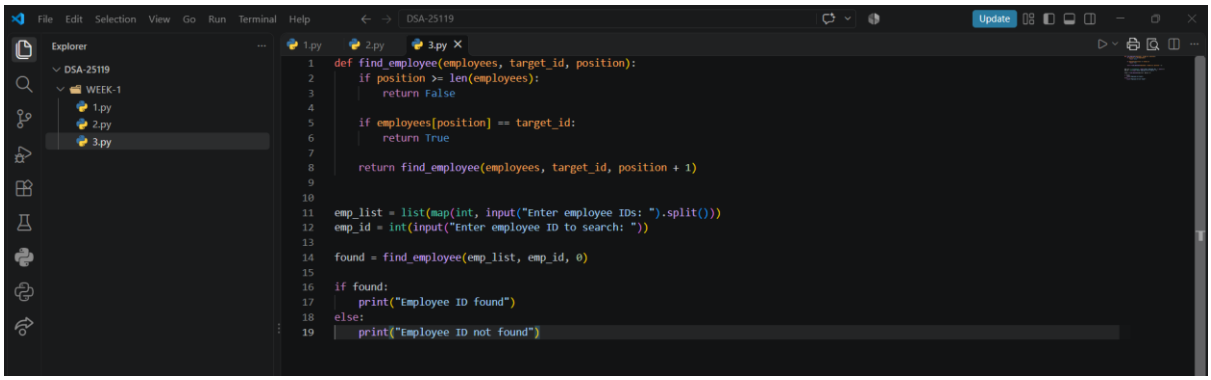
- The recursive calls then return one by one.
- Each returned value is multiplied by p.
- Finally, the result is printed using print(compound_interest(p,n))

Error table:

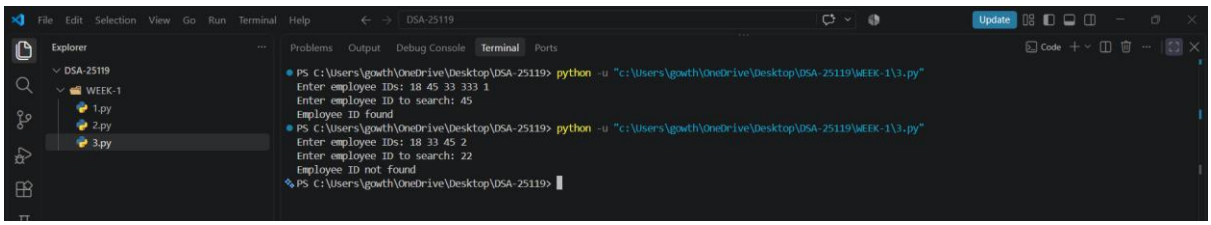
s.no	Error obtained	rectification
1	Wrong indentation	Maintain proper indentation.

1C. AIM . An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code:



Output:



Explanation :

Condition 1: index == len(emp_list)

- The function checks whether index has reached the length of the employee list.
- If index == len(emp_list), it means we have checked **all employee IDs**.
- The employee ID was not found.

- The function returns **False**.
- No further recursion takes place.

Condition 2: emp_list[index] == emp_id

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

Condition 3: Employee ID does not match

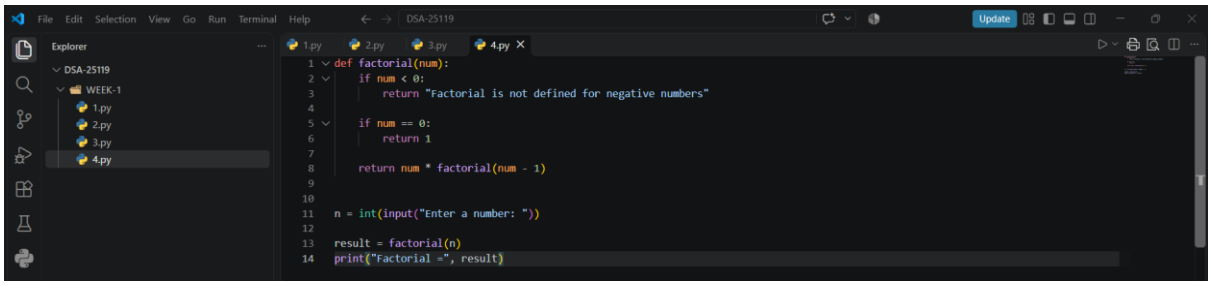
- If the current employee ID is not equal to emp_id, the condition is **False**.
- The function calls itself recursively with:
  - index + 1
- This moves to the **next employee ID** in the list.
- The process continues until:
  - The employee ID is found, or
  - The entire list is checked.

Error table:

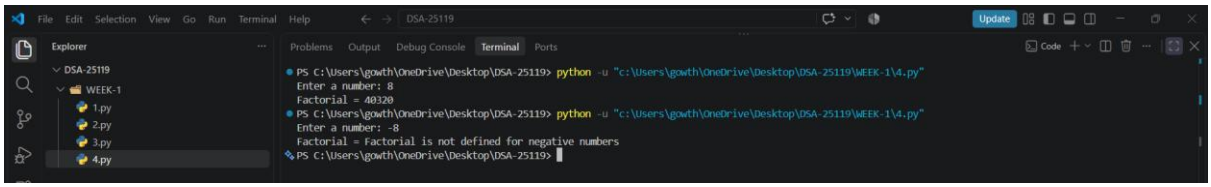
s.no	Error obtained	Rectification
1	Missing return statement	Add the required return statement

1D. AIM. In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code:



Output:



Explanation :

Condition 1: Negative Input (n < 0)

- The program first checks whether n < 0.
- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
  Factorial is not defined for negative numbers.
- The factorial() function is **not called**.

Condition 2: Base Case (n == 0 or n == 1)

- Inside the factorial() function, it checks:
  n == 0 or n == 1
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- This is the **base case** of the recursion.
- It stops further recursive calls.

Condition 3: n > 1

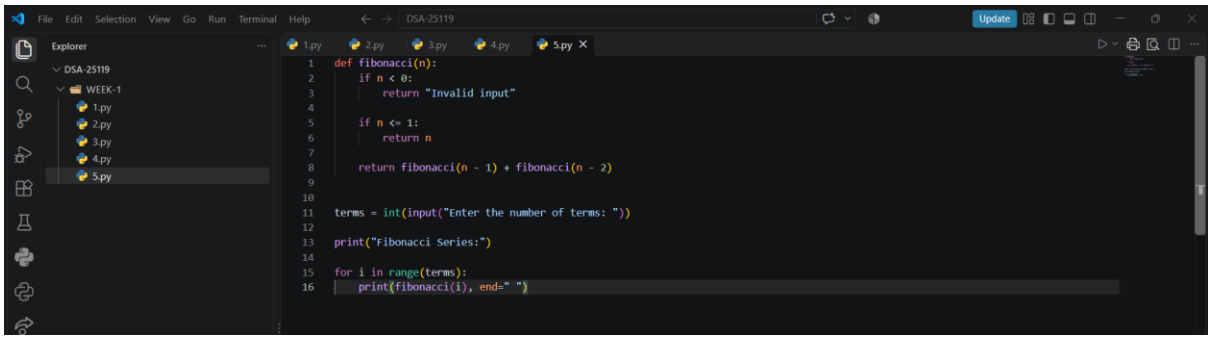
- If n is greater than 1, the condition n == 0 or n == 1 is **False**.
- The else block is executed.
- It returns:
  n * factorial(n - 1)
- The function calls itself with n - 1.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

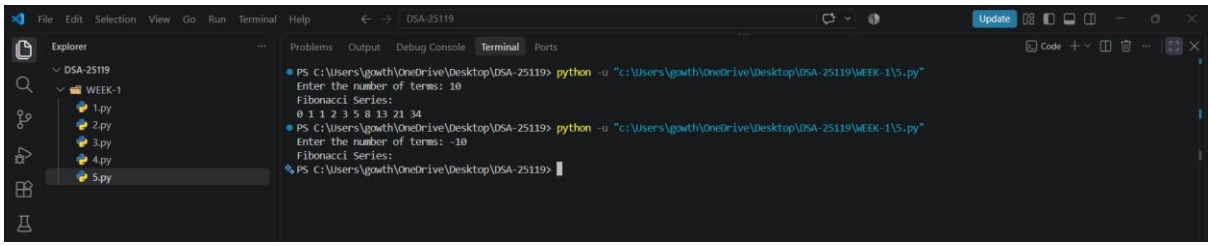
s.no	Error obtained	Rectification
1	Wrong recursive value	Updated the recursive value correctly

1E. AIM. Write a recursive program to print the first n terms of the fibonacci series

Code:



Output:



Explanation :

Condition 1: n == 0

- The function checks whether n == 0.
- If the condition is **True**, the function returns 0.

- This is the **first base case**.
- No further recursive call takes place.

Condition 2: n == 1

- If n is not 0, the function checks whether n == 1.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.
- No further recursive call takes place

Condition 3: n > 1

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.
- The function calls itself **two times**:
  - fibonacci(n - 1)
  - fibonacci(n - 2)
- The two returned values are added together.
- This process continues until the base cases n == 0 and n == 1 are reached.

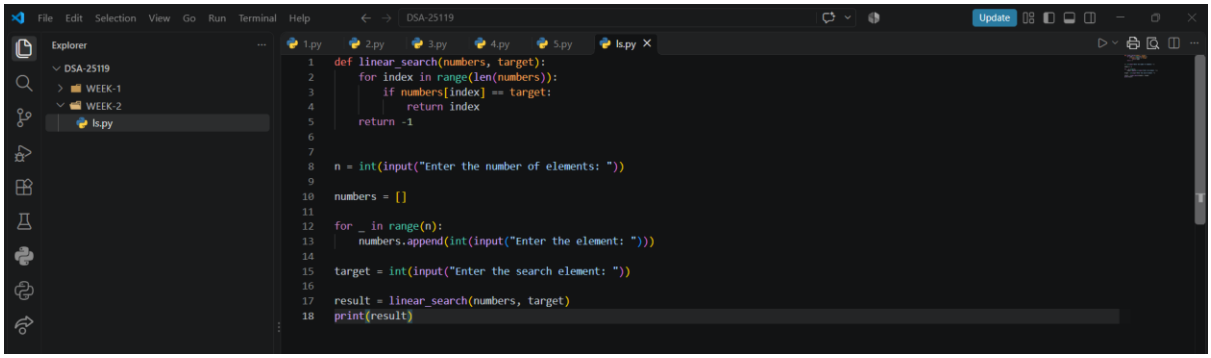
Error table:

S.no	Error obtained	Rectification
1	Incorrect Fibonacci recursion	Use fibonacci(n-1) + fibonacci(n-2).

Week-2

2a. AIM: Write a python program to perform linear search on the give input.

Code:



```python
def linear_search(numbers, target):
    for index in range(len(numbers)):
        if numbers[index] == target:
            return index
    return -1


n = int(input("Enter the number of elements: "))

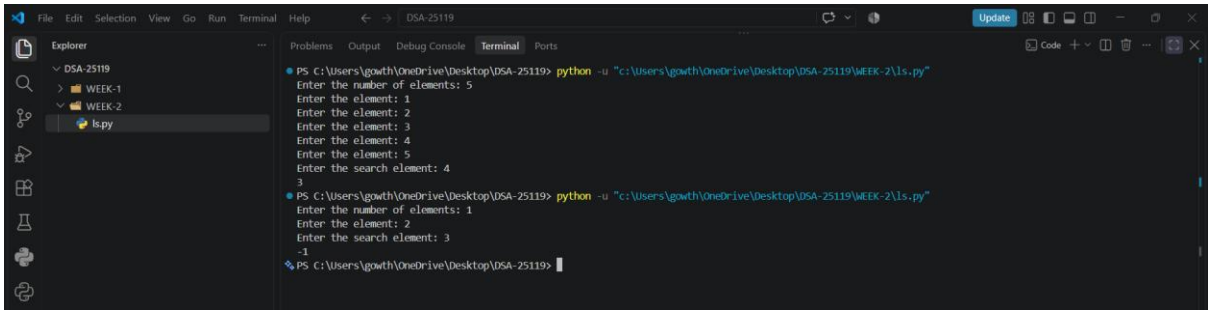
numbers = []

for _ in range(n):
    numbers.append(int(input("Enter the element: ")))

target = int(input("Enter the search element: "))

result = linear_search(numbers, target)
print(result)
```

Output:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-2\ls.py"
Enter the number of elements: 5
Enter the element: 1
Enter the element: 2
Enter the element: 3
Enter the element: 4
Enter the element: 5
Enter the search element: 4
3
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-2\ls.py"
Enter the number of elements: 1
Enter the element: 2
Enter the search element: 3
-1
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

Explanation :

Condition 1: Element is Found (arr[i] == k)

- The function linear_search(arr, k) receives the array arr and the search element k.
- The for loop starts from the first element of the array.
- It checks each element one by one using:
  arr[i] == k
- If the current element is equal to k, the condition becomes **True**.
- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

Condition 2: Element is Not Found

- The loop checks every element in the array.
- If none of the elements is equal to k, the if condition is always **False**.

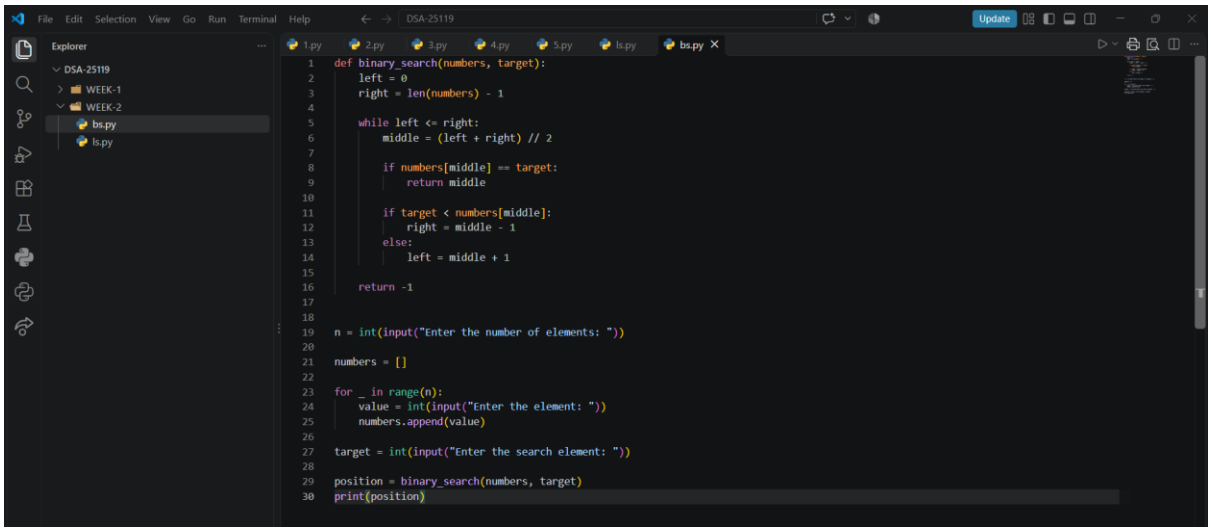
- After the loop finishes, the statement return -1 is executed.
- -1 indicates that the search element is **not present** in the array.

Error table:

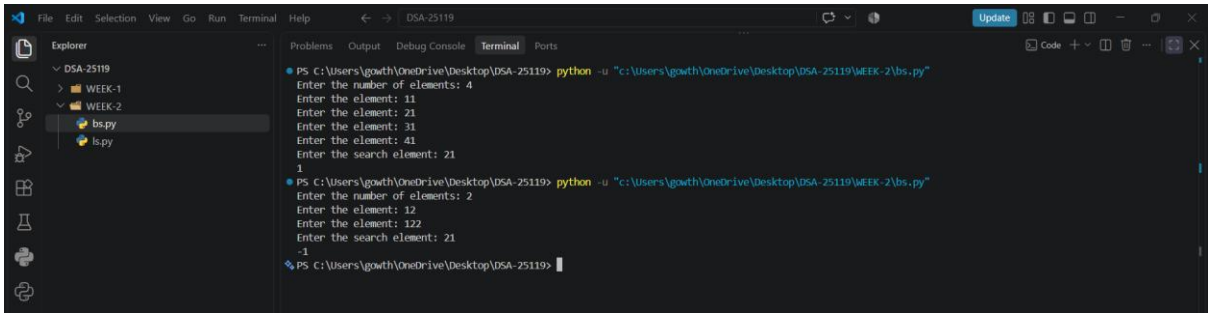
s.no	Error obtained	Rectification
1	Wrong loop range	Use range(n) to process exactly n elements.
2	Element not found	Return -1 when the search element is not present in the list.

2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

Code:



Output:



Explanation :

Condition 1: k == arr[mid] — Element Found

- The function binary_search(arr, k) receives the array and the search element.

- low is initialized to 0, which represents the first index.
- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
  mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

Condition 2: k < arr[mid] — Search Left Half

- If k is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
  high = mid - 1
- The right half is eliminated.
- The process continues with the remaining left half

Condition 3: k > arr[mid] — Search Right Half

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.
- This means the search element can only be present in the **right half**.
- Therefore:
  low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

Condition 4: low > high — Element Not Found

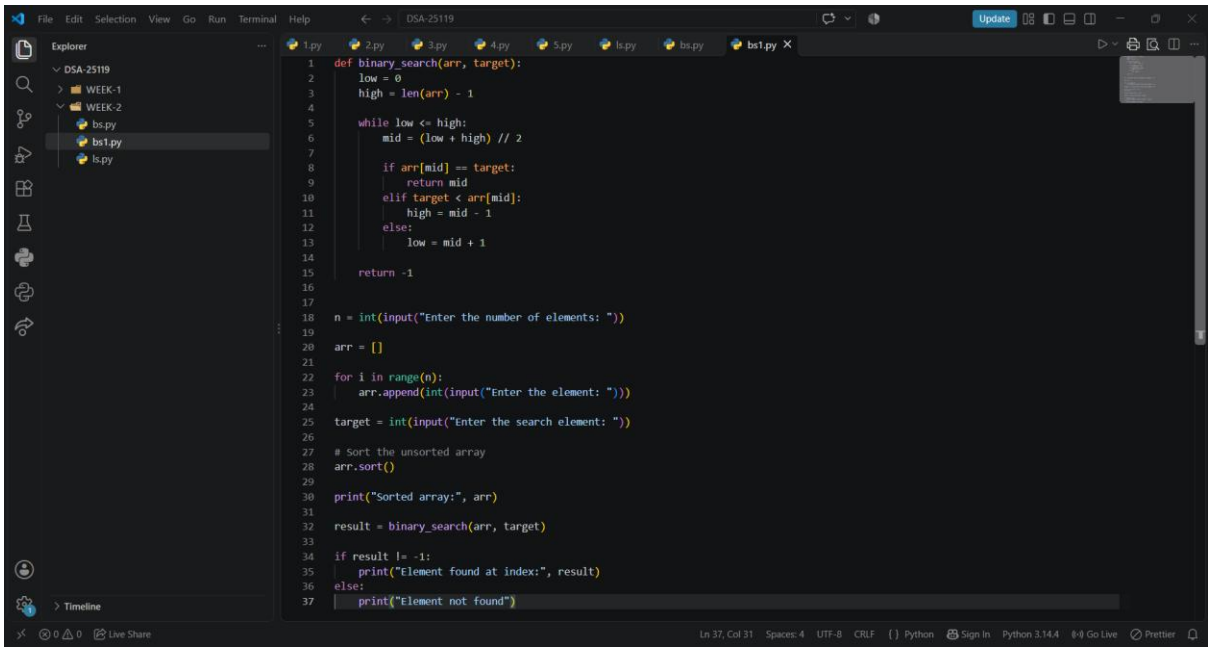
- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

S.no	Error obtained	Rectification
1.	Wrong index in binary search	Use high = mid - 1 for the left half and low = mid + 1 for the right half.

2c.Aim: write a python program to perform a bianry search on the given input verify by Taking unsorted array as input

Code:



```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif target < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1


n = int(input("Enter the number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input("Enter the element: ")))

target = int(input("Enter the search element: "))

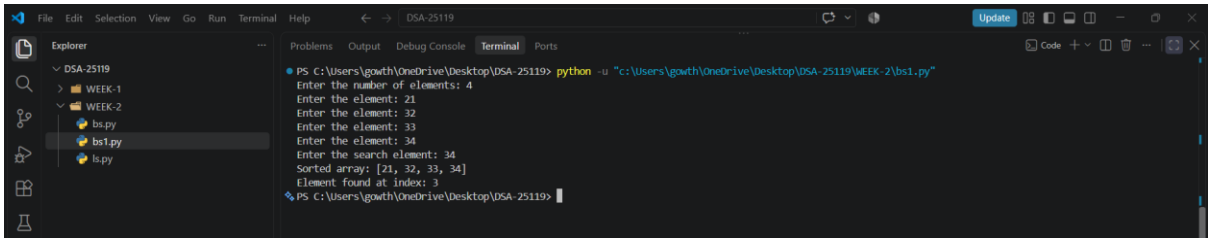
# Sort the unsorted array
arr.sort()

print("Sorted array:", arr)

result = binary_search(arr, target)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Output:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-2\bs1.py"
Enter the number of elements: 4
Enter the element: 21
Enter the element: 32
Enter the element: 33
Enter the element: 34
Enter the search element: 34
Sorted array: [21, 32, 33, 34]
Element found at index: 3
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

Explanation :

Condition 1: arr[mid] == key — Element Found

- The function first calculates the middle index using mid = (low + high) // 2.
- It compares the middle element arr[mid] with the search element key.

- If arr[mid] == key, the condition is **True**.
- The element is found.
- The function returns the index mid.
- The search stops.

Condition 2: key < arr[mid] — Search Left Half

- If key is smaller than arr[mid], the condition is **True**.
- This means the required element may be present in the **left half**.
- The right half is ignored.
- high is changed to mid - 1.
- The search continues in the left half.

Condition 3: key > arr[mid] — Search Right Half

- If key is greater than arr[mid], the previous conditions are **False**.
- The element may be present in the **right half**.
- The left half is ignored.
- low is changed to mid + 1.
- The search continues in the right half.

Condition 4: low > high — Element Not Found

- The while low <= high loop continues until there are elements left to search.
- When low > high, the loop stops.
- This means the element is not present in the array.
- The function returns -1.
- The program prints **"Element not found."**

Condition 5: Array is Already Sorted

- The program checks whether the given array is already sorted.
- It uses:
  arr == sorted(arr)
- If the condition is **True**, it prints:
  "Array is already sorted."
- The array is directly used for binary search.

Condition 6: Array is Unsorted

- If arr == sorted(arr) is **False**, the array is unsorted.
- The program prints:
  "Array is unsorted."
- It then uses arr.sort() to sort the array.
- The sorted array is displayed.
- Binary search is then performed on the sorted array.

Error table:

s.no	Error obtained	Rectification
1	Binary search on unsorted array	sort the array first using arr.sort() or provide a sorted array.

Week-3

3a.Aim.Write a python program to sort the elements using a bubble sort technique. Take an input elements from the user.

Code:



```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


n = int(input("Enter the number of elements: "))

arr = []

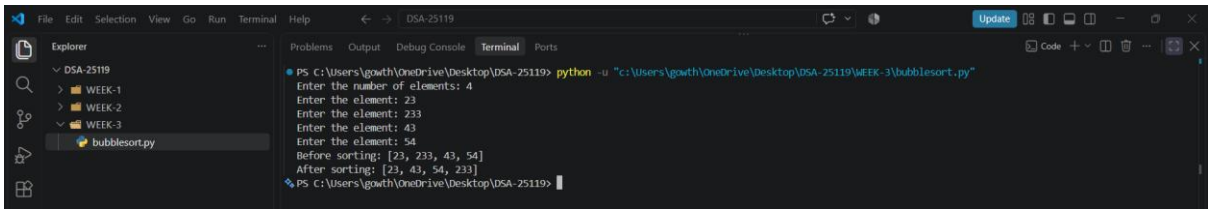
for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

bubble_sort(arr)

print("After sorting:", arr)
```

Output:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-3\bubblesort.py"
Enter the number of elements: 4
Enter the element: 23
Enter the element: 233
Enter the element: 43
Enter the element: 54
Before sorting: [23, 233, 43, 54]
After sorting: [23, 43, 54, 233]
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

Explanation :

Condition 1: arr[j] > arr[j+1] — Elements Need to Be Swapped

- The function bubble_sort(arr) receives the array as an argument.
- The outer for loop runs from 0 to n-1 and controls the number of passes.
- The inner for loop runs from 0 to n-2 and compares adjacent elements.
- The condition arr[j] > arr[j+1] checks whether the current element is greater than the next element.
- If the condition is **True**, the two elements are in the wrong order.
- Therefore, the two elements are swapped using:
- Arr[j],arr[j+1]=arr[j+1],arr[j]
- After the swap, the smaller element moves toward the left and the larger element moves toward the right.

Condition 2: arr[j] <= arr[j+1] — No Swap Required

- If arr[j] > arr[j+1] is **False**, it means the current element is smaller than or equal to the next element.

- Therefore, the elements are already in the correct ascending order.
- No swapping is performed.
- The inner loop continues to compare the next pair of adjacent elements.

Condition 3: for i in range(n) — Repeated Passes

- The outer loop repeats the comparison process for all the elements.
- After each complete pass, the **largest unsorted element moves to the end of the array**.
- The process is repeated until all elements are arranged in ascending order.
- Finally, the function returns the sorted array using:

Error table:

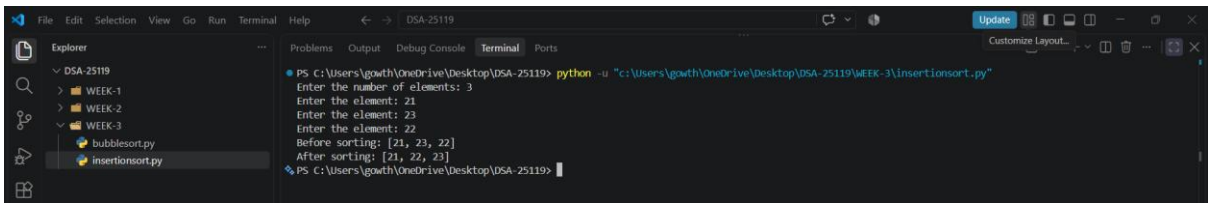
sno	Error obtained	Rectification
1	IndexError: list index out of range	Use correct loop range

3b.Aim. write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code:



Output:



Explanation:

Condition 1: for i in range(1, len(arr)) — Selecting Element

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

Condition 2: key = arr[i] — Storing Element

- The current element is stored in key.

```
key = arr[i]
```

- This element will be compared with the elements before it.

Condition 3: j = i - 1 — Getting Previous Position

- j points to the element just before the current element.

```
j = i - 1
```

Condition 4: while j >= 0 and key < arr[j] — Comparing Elements

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

```
while j >= 0 and key < arr[j]:
    arr[j + 1] = arr[j]
    j -= 1
```

- This creates space for the current element.

Condition 5: arr[j + 1] = key — Inserting Element

- The key is placed in its correct position.

```
arr[j + 1] = key
```

- The elements before the current position are now sorted.

Condition 6: return arr — Returning Sorted Array

- After all elements are processed, the function returns the sorted array.

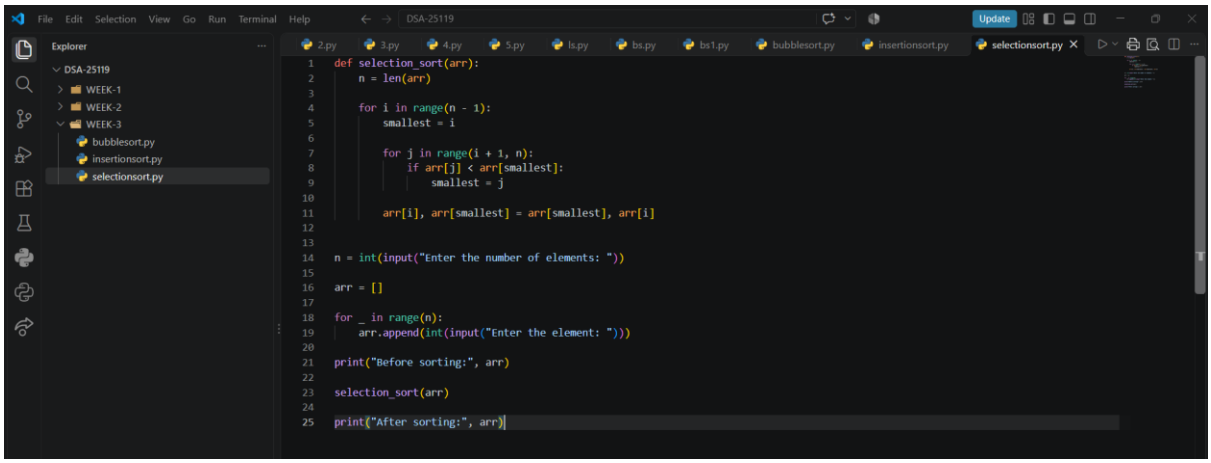
```
return arr
```

Error table:

sno	Error obtained	Rectification
1	Elements not sorted correctly	Shift elements before insertion

3c.write a python program to sort the elements using selection sort technique . take an input from the user.

Code:



```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        smallest = i

        for j in range(i + 1, n):
            if arr[j] < arr[smallest]:
                smallest = j

        arr[i], arr[smallest] = arr[smallest], arr[i]


n = int(input("Enter the number of elements: "))

arr = []

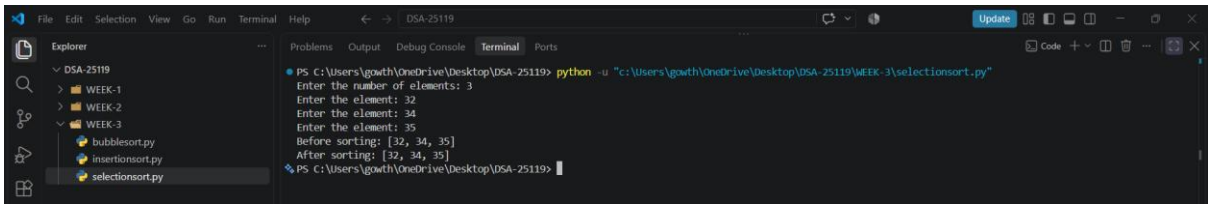
for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

selection_sort(arr)

print("After sorting:", arr)
```

Output:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-3\selectionsort.py"
Enter the number of elements: 3
Enter the element: 32
Enter the element: 34
Enter the element: 35
Before sorting: [32, 34, 35]
After sorting: [32, 34, 35]
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

Explanation:

Condition 1: for i in range(n) — Selecting Position

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

Condition 2: mini = i — Assuming Minimum

- Initially, the element at index i is assumed to be the smallest.

Condition 3: for j in range(i+1,n) — Finding Minimum

- The loop checks all elements after position i.
- It searches for the smallest element.

Condition 4: if arr[mini] > arr[j] — Comparing

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

Condition 5: mini = j — Updating Minimum

- Stores the index of the newly found smallest element.

Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

Condition 7: return arr — Returning Result

- Returns the completely sorted array.

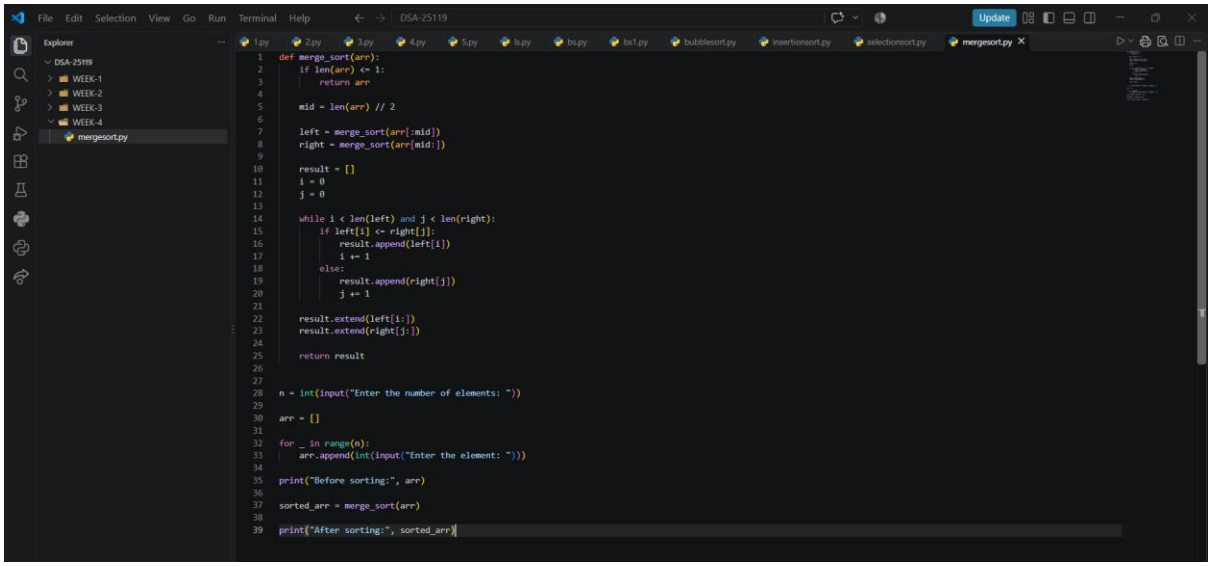
Error table:

Sno	Error obtained	Rectification
1	Wrong minimum value	Update min_index properly

WEEK -4

4a.write a python program to sort the elements using the merge sort technique . take an input from the user.

Code:



```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


n = int(input("Enter the number of elements: "))

arr = []

for _ in range(n):
    arr.append(int(input("Enter the element: ")))

print("Before sorting:", arr)

sorted_arr = merge_sort(arr)

print("After sorting:", sorted_arr)
```

Output:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-4\mergesort.py"
Enter the number of elements: 4
Enter the element: 11
Enter the element: 21
Enter the element: 31
Enter the element: 41
Before sorting: [11, 21, 31, 41]
After sorting: [11, 21, 31, 41]
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

Explanation:

Condition 1: if len(arr) <= 1 — Checking the Base Condition

- The function first checks whether the array contains **one or zero elements**.

if len(arr) <= 1:

return arr

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

return arr

Condition 2: mid = len(arr) // 2 — Finding the Middle

- The array is divided into two parts.
- The middle position is calculated using:

mid = len(arr) // 2

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

Condition 3: left = arr[:mid] — Creating the Left Part

- The elements before the middle position are stored in left.

left = arr[:mid]

Condition 4: right = arr[mid:] — Creating the Right Part

- The elements from the middle position to the end are stored in right.

right = arr[mid:]

Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts

- The left and right parts are recursively sorted.

merge_sort(left)

merge_sort(right)

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

Condition 6: i = j = k = 0 — Initializing Indexes

- Three variables are initialized:

i = j = k = 0

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts

- The program compares elements from the left and right arrays.

while i < len(left) and j < len(right):

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

Condition 8: if left[i] < right[j] — Comparing Elements

- The current elements of the left and right arrays are compared.

```
if left[i] < right[j]:
```

- If the left element is smaller than the right element, it is placed into the original array.

```
arr[k] = left[i]
```

- Then i is increased:

```
i += 1
```

- This moves to the next element of the left array.

Condition 9: else — Taking the Right Element

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

```
else:
    arr[k] = right[j]
```

- Then j is increased:

```
j += 1
```

- This moves to the next element of the right array.

Condition 10: k += 1 — Moving to the Next Position

- After placing an element into the original array, k is increased.

```
k += 1
```

Condition 11: while i < len(left) — Copying Remaining Left Elements

- Sometimes the right array becomes empty before the left array.

```
while i < len(left):
```

- Any remaining elements in the left array are copied into the original array.

```
arr[k] = left[i]
```

- Then both i and k are increased:

```
i += 1
k += 1
```

- The remaining elements are already sorted, so no comparison is needed.

Condition 12: while j < len(right) — Copying Remaining Right Elements

- Sometimes the left array becomes empty before the right array.

while j < len(right):

- Any remaining elements in the right array are copied into the original array.

arr[k] = right[j]

- Then j and k are increased:

j += 1

k += 1

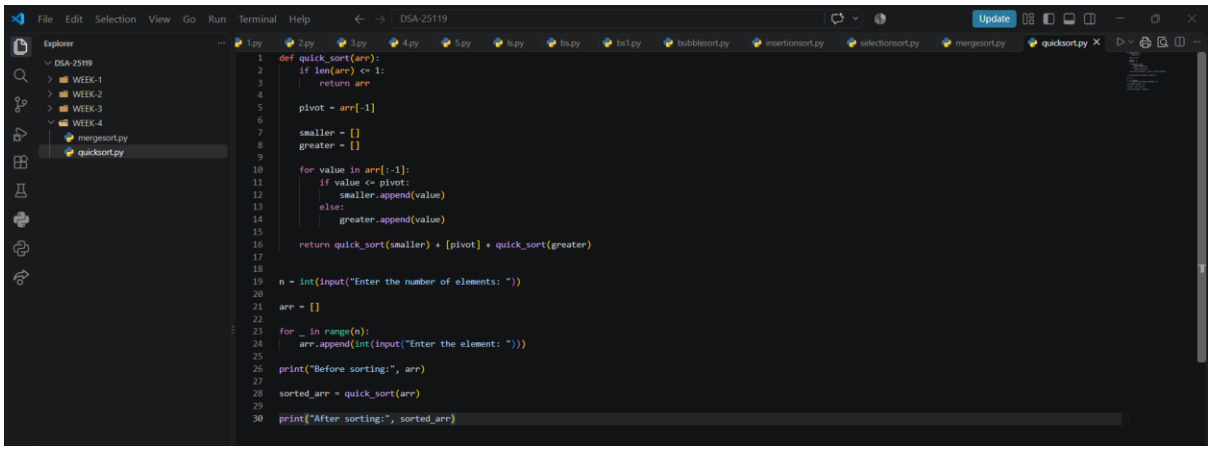
- This completes the merging process.

Error table:

sno	Error obtained	Rectification
1	Recursive error	Use if len(arr) <= 1
2	IndexError	Check left/right array indices

4b. Aim. Write a python program to sort the elements using a quick sort technique . take an input elements from the user

Code:



Output:



Explanation :

Condition 1: if arr[j] < pivot — Element Smaller Than Pivot

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.
- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.
- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

arr[i], arr[j] = arr[j], arr[i]

- This places all elements smaller than the pivot toward the left side.

Condition 2: arr[j] >= pivot — No Swap Required

- If arr[j] < pivot is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

Condition 3: arr[i+1], arr[high] = arr[high], arr[i+1] — Placing the Pivot

- After all elements have been checked, the pivot is placed in its correct position.
- The pivot is swapped with the element at index i+1:

arr[i+1], arr[high] = arr[high], arr[i+1]

- After this operation:
  - Elements smaller than the pivot are on the **left**.
  - The pivot is in its **correct position**.
  - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

return i+1

Condition 4: if low < high — Continue Quick Sort

- The function quick_sort(arr, low, high) is used to sort the array.

- The condition low < high checks whether the current part of the array contains more than one element.
- If the condition is **True**, the array needs to be sorted.
- The partition() function is called:

pi = partition(arr, low, high)

- pi stores the final position of the pivot.

Condition 5: quick_sort(arr, low, pi-1) — Sort Left Part

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, low, pi-1)

- The pivot itself is not included because it is already in its correct position.

Condition 6: quick_sort(arr, pi+1, high) — Sort Right Part

- The elements on the right side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, pi+1, high)

- Again, the pivot is not included because it is already correctly positioned.

Input Section

- n = int(input("enter the no of elements:")) takes the number of elements from the user.
- arr = [] creates an empty list to store the elements.
- The for loop runs n times and takes each element from the user:
- for i in range(n):

arr.append(int(input("enter the elements:")))

- Each entered value is converted into an integer and added to the array using append().

Calling the Quick Sort Function

quick_sort(arr, 0, n-1)

- This calls the quick_sort() function.
- 0 represents the first index of the array.
- n-1 represents the last index of the array.

- The complete array is therefore passed for sorting.

.

Error table:

sno	Error obtained	Rectification
1	IndexError	Check i, j, and pivot boundaries
2	Wrong swapping conditions	By shifting to the correct spaces

WEEK-5

5.(A)AIM: write a python program to implement a single linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

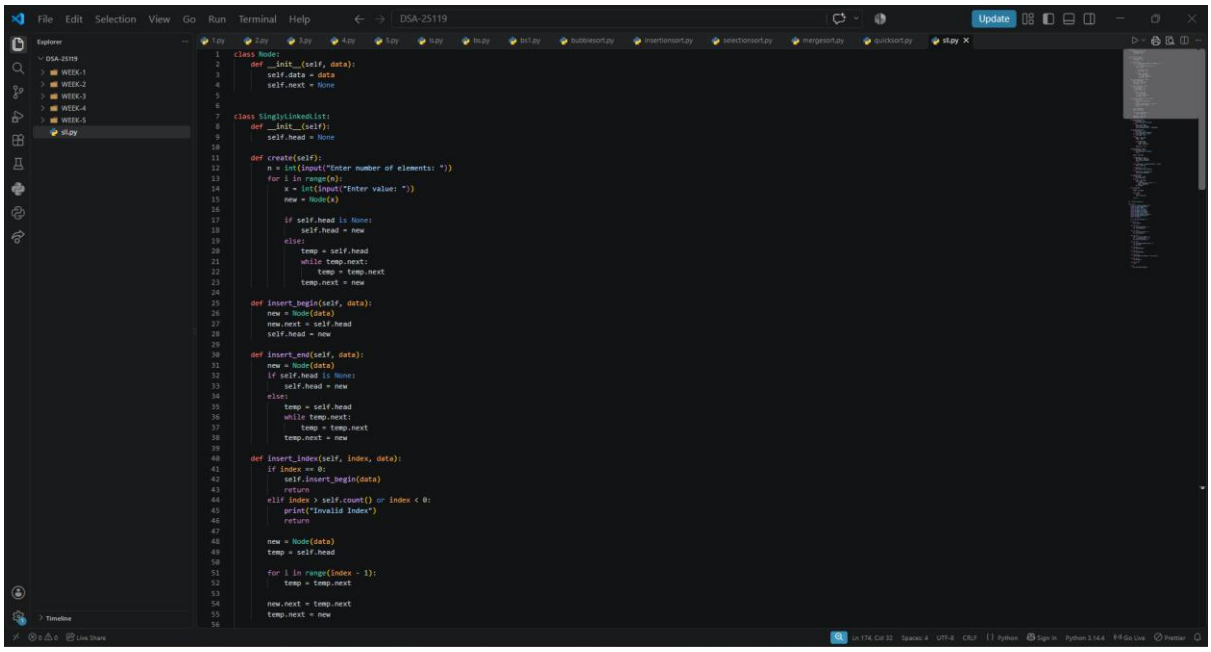
vii. delete at the end

viii. count no of nodes

ix. display or travsere

x. exit

Code:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of elements: "))
        for i in range(n):
            x = int(input("Enter value: "))
            new = Node(x)

            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

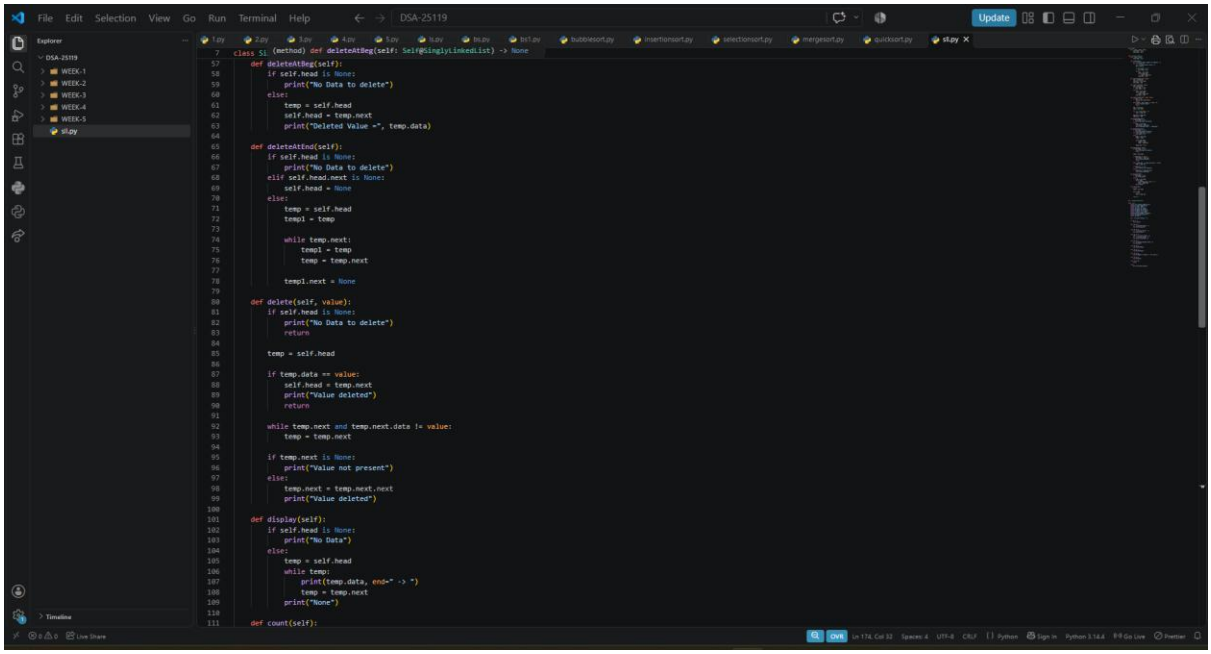
    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def insert_index(self, index, data):
        if index == 0:
            self.insert_begin(data)
            return
        elif index > self.count() or index < 0:
            print("Invalid Index")
            return

        new = Node(data)
        temp = self.head

        for i in range(index - 1):
            temp = temp.next

        new.next = temp.next
        temp.next = new
```



```
class SL (method) def deleteAtBeg(self: Self@SinglyLinkedList) -> None
    def deleteAtBeg(self):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted Value =", temp.data)

    def deleteAtEnd(self):
        if self.head is None:
            print("No Data to delete")
        elif self.head.next is None:
            self.head = None
        else:
            temp = self.head
            temp1 = temp

            while temp.next:
                temp1 = temp
                temp = temp.next

            temp1.next = None

    def delete(self, value):
        if self.head is None:
            print("No Data to delete")
            return

        temp = self.head

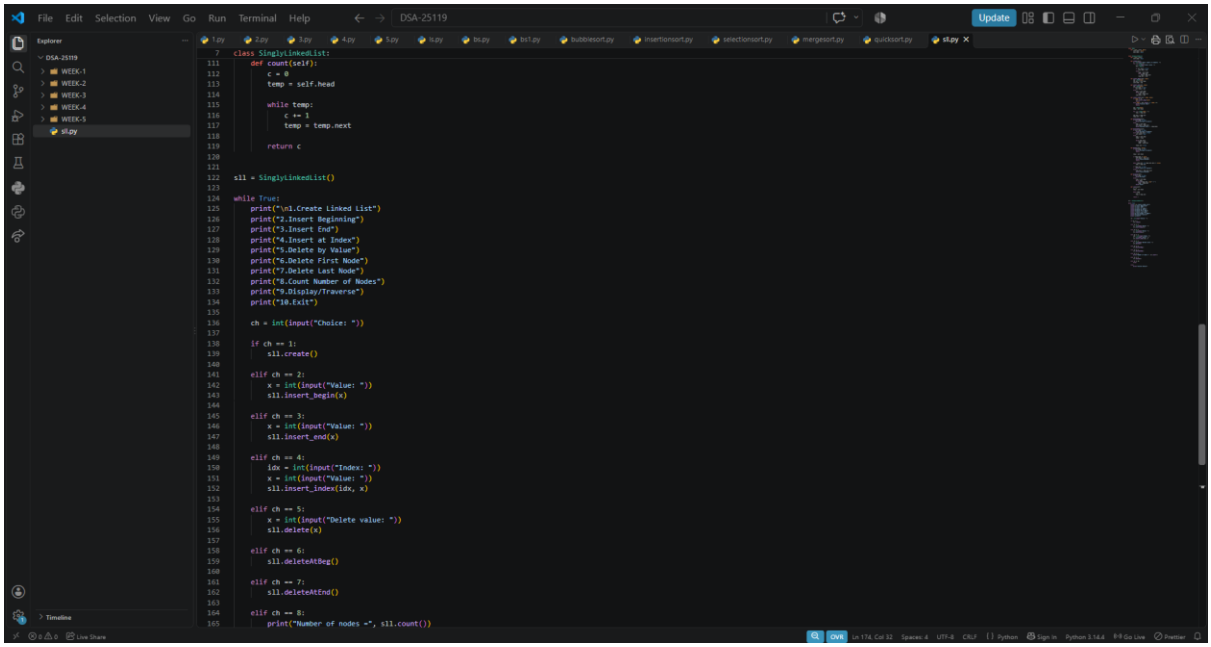
        if temp.data == value:
            self.head = temp.next
            print("Value deleted")
            return

        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")

    def display(self):
        if self.head is None:
            print("No Data")
        else:
            temp = self.head
            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

    def count(self):
```



```
class SinglyLinkedList:
    def count(self):
        c = 0
        temp = self.head

        while temp:
            c += 1
            temp = temp.next

        return c


sll = SinglyLinkedList()

while True:
    print("\n1.Create Linked List")
    print("2.Insert Beginning")
    print("3.Insert End")
    print("4.Insert at Index")
    print("5.Delete by Value")
    print("6.Delete First Node")
    print("7.Delete Last Node")
    print("8.Count Number of Nodes")
    print("9.Display/Traverse")
    print("10.Exit")

    ch = int(input("Choice: "))

    if ch == 1:
        sll.create()

    elif ch == 2:
        x = int(input("Value: "))
        sll.insert_begin(x)

    elif ch == 3:
        x = int(input("Value: "))
        sll.insert_end(x)

    elif ch == 4:
        idx = int(input("Index: "))
        x = int(input("Value: "))
        sll.insert_index(idx, x)

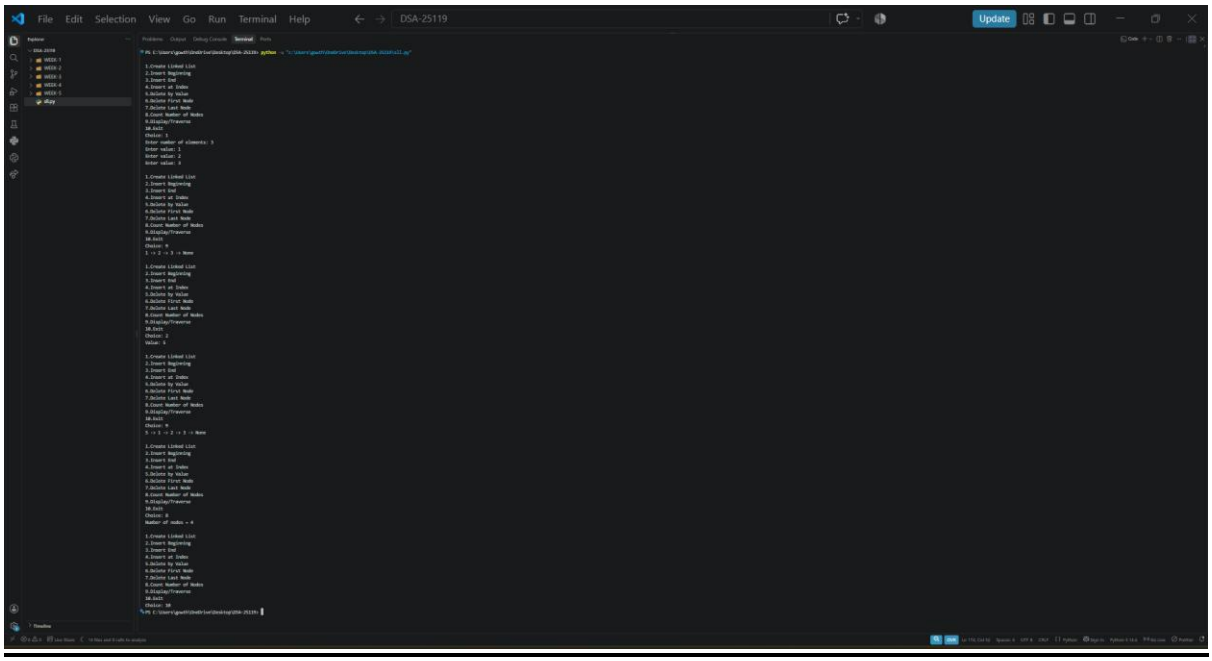
    elif ch == 5:
        x = int(input("Delete value: "))
        sll.delete(x)

    elif ch == 6:
        sll.deleteAtBeg()

    elif ch == 7:
        sll.deleteAtEnd()

    elif ch == 8:
        print("Number of nodes =", sll.count())
```

OUTPUT:



Explanation:

1. A **singly linked list** is a collection of nodes where each node contains **two parts**: data and a link to the next node.
2. Each node contains:
   - **Data** – stores the value.
   - **Next (next)** – stores the reference/address of the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are created one by one and added to the list.
7. For the first node, both **head and tail** point to the same node, and the next of the tail is None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node's next points to the current head, and the new node becomes the new head.

9. During **insertion at the end**, the new node is added after the current tail. The current tail's next points to the new node, and the new node becomes the new tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the appropriate nodes by adjusting their next links.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by making the previous node point to the node after the deleted node.
12. If the node to be deleted is the **head**, the next node becomes the new head.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
16. During **deletion at the end**, the list is traversed to find the node before the tail. That node becomes the new tail and its next is set to None.
17. The **count operation** traverses the list from the head to the end and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves through the list using the next link until None is reached.
20. The program continues to execute until the user selects the **Exit** option.

Error table:

sno	Error obtained	rectification
1	Infinite loop or incomplete traversal due to not updating current	Move to the next node using temp = temp.next
2	Forgetting to update head while inserting/deleting at the beginning	Update head to point to the new first node using head = new_node or head = head.next
3	Element not found during deletion.	Check if temp.next is None and display "Value not present".

5.(B)AIM: write a python program to implement a double linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

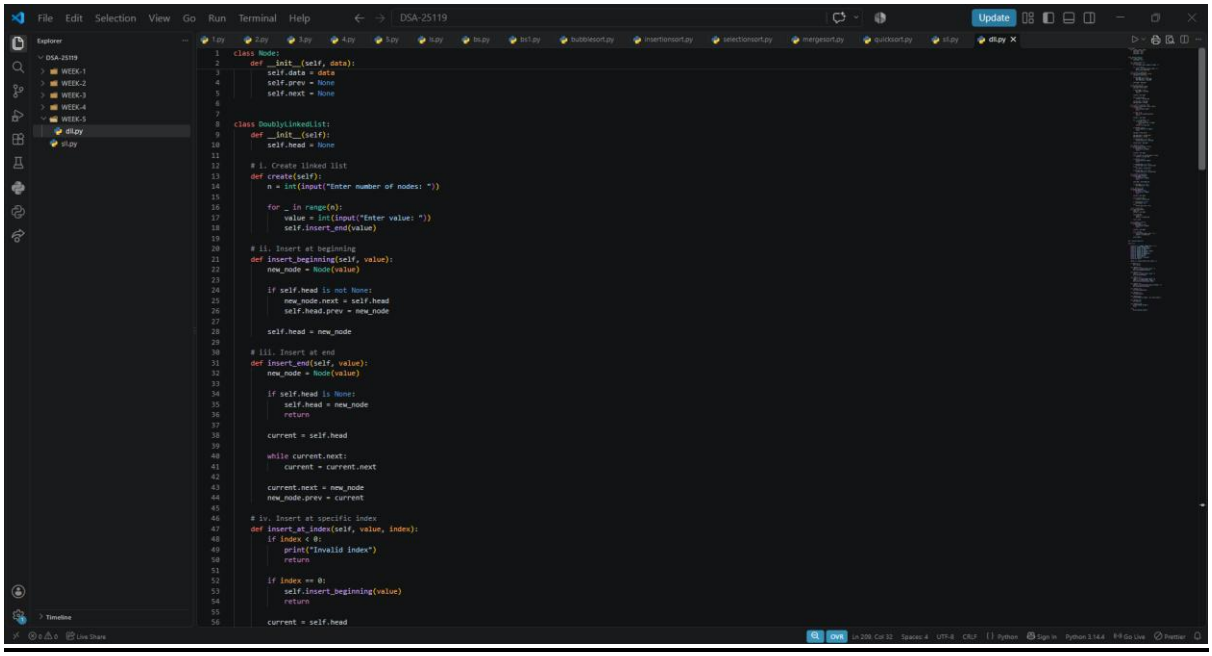
vii. delete at the end

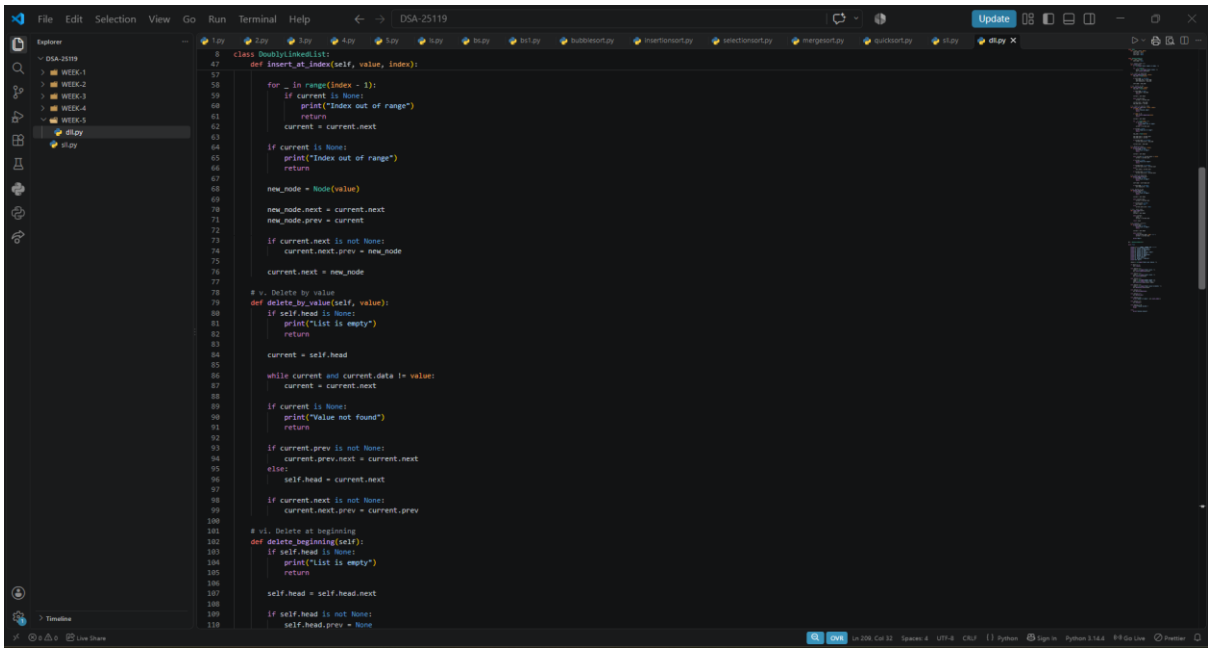
viii. count no of nodes

ix. display or travsere

x. exit

code:





```
class DoublyLinkedList:
    def insert_at_index(self, value, index):

        for _ in range(index - 1):
            if current is None:
                print("Index out of range")
                return
            current = current.next

        if current is None:
            print("Index out of range")
            return

        new_node = Node(value)

        new_node.next = current.next
        new_node.prev = current

        if current.next is not None:
            current.next.prev = new_node

        current.next = new_node

    # v. Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current and current.data != value:
            current = current.next

        if current is None:
            print("Value not found")
            return

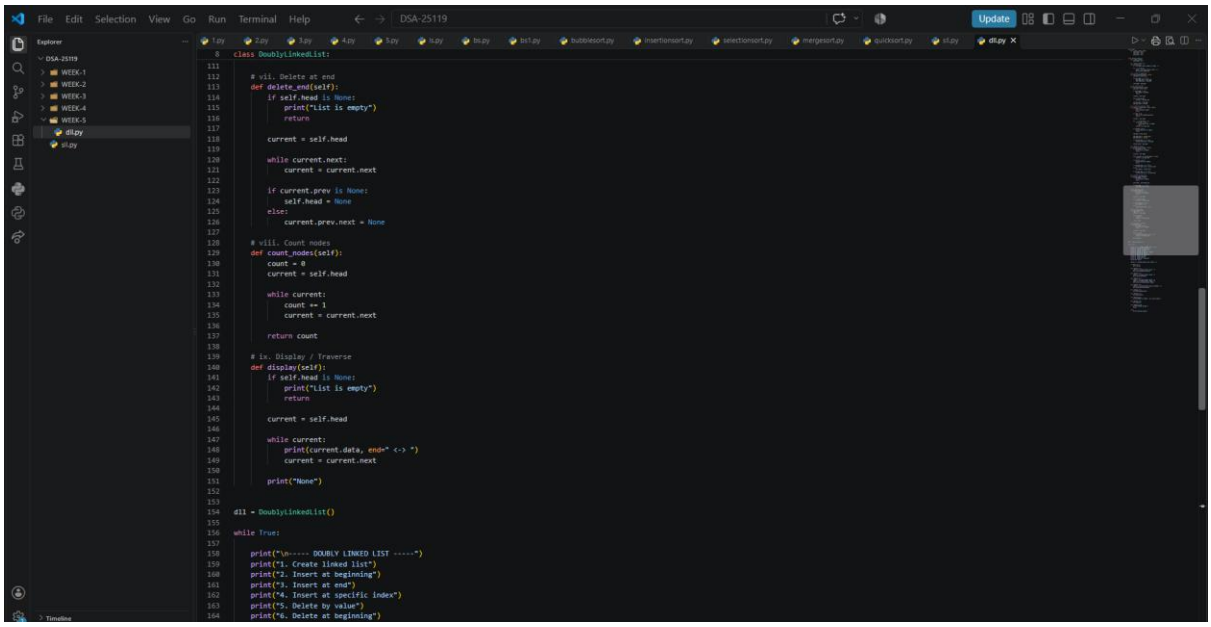
        if current.prev is not None:
            current.prev.next = current.next
        else:
            self.head = current.next

        if current.next is not None:
            current.next.prev = current.prev

    # vi. Delete at beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None
```



```
class DoublyLinkedList:

    # vii. Delete at end
    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current.next:
            current = current.next

        if current.prev is None:
            self.head = None
        else:
            current.prev.next = None

    # viii. Count nodes
    def count_nodes(self):
        count = 0
        current = self.head

        while current:
            count += 1
            current = current.next

        return count

    # ix. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current:
            print(current.data, end=" <-> ")
            current = current.next

        print("None")


dll = DoublyLinkedList()

while True:

    print("\n---- DOUBLY LINKED LIST ----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
```

```
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        value = int(input("Enter value: "))
        dll.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        dll.insert_end(value)

    elif choice == 4:
        value = int(input("Enter value: "))
        index = int(input("Enter index: "))
        dll.insert_at_index(value, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        dll.delete_by_value(value)

    elif choice == 6:
        dll.delete_beginning()

    elif choice == 7:
        dll.delete_end()

    elif choice == 8:
        print("Number of nodes:", dll.count_nodes())

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

Output:

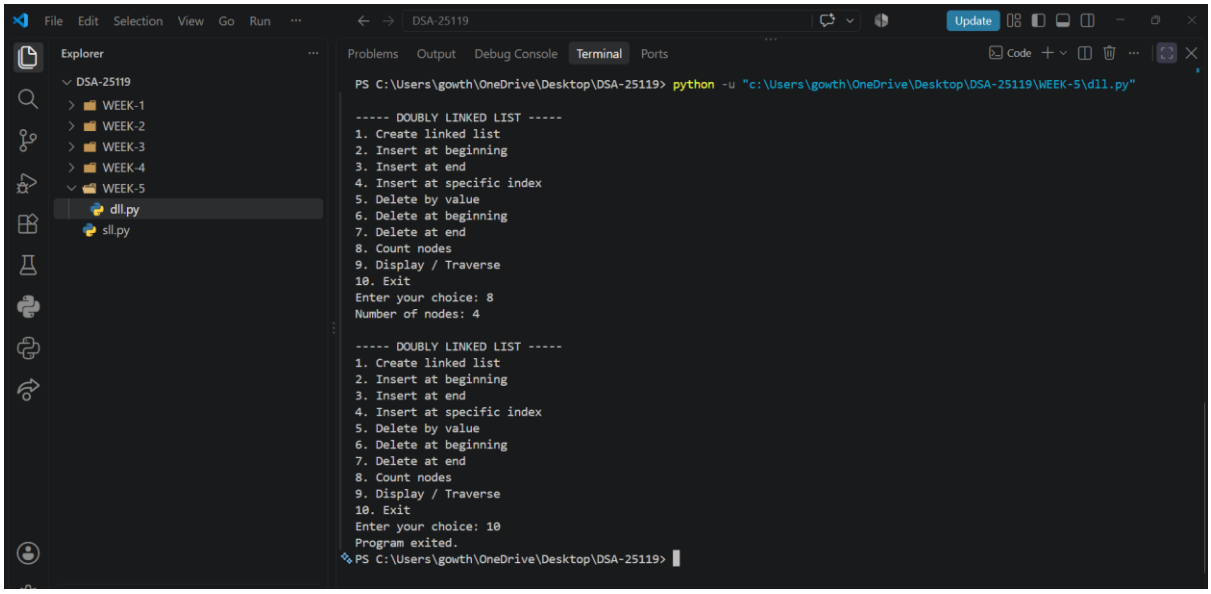
```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-5\dll.py"

----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter value: 2
Enter value: 3
Enter value: 4

----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
2 <-> 3 <-> 4 <-> None

----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter value: 5

----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
5 <-> 2 <-> 3 <-> 4 <-> None
```



Explanation:

1. A **doubly linked list** is a collection of nodes where each node contains **three parts**: data, a link to the previous node, and a link to the next node.
2. Each node has:
   - **Data** – stores the value.
   - **Previous (prev)** – points to the previous node.
   - **Next (next)** – points to the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and maintain the data, prev, and next references.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are added one by one at the end of the list.
7. For the first node, both **head and tail** point to the same node. The prev of the head and the next of the tail are None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node becomes the head, its next points to the previous head, and the previous head's prev points back to the new node.

9. During **insertion at the end**, the new node is placed after the current tail. The new node's prev points to the old tail, the old tail's next points to the new node, and the new node becomes the tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the previous and next nodes, and both prev and next links are updated.
11. During **deletion by value**, the list is searched for the node containing the given value. Once found, its previous and next nodes are connected to each other, removing the selected node from the list.
12. If the node to be deleted is the **head**, the next node becomes the new head and its prev is set to None.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head. The new head's prev is set to None.
16. During **deletion at the end**, the tail node is removed and the previous node becomes the new tail. The new tail's next is set to None.
17. The **count operation** traverses the list from the head to the tail and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves forward using the next link to display all the nodes.
19. Since it is a doubly linked list, the program can also traverse the list **backwards**, starting from the tail and using the prev links.
20. The program displays the **head and tail** values to show the starting and ending nodes of the list.
21. The important relationships maintained in a doubly linked list are:

head.prev = None

tail.next = None

22. The overall structure of a doubly linked list is:

None ← [10] ⇄ [20] ⇄ [30] ⇄ [40] → None

↑ ↑

HEAD TAIL

23. The arrows in both directions allow the list to be traversed **forward as well as backward**.
24. The program is **menu-driven**, allowing the user to perform creation, insertion, deletion, counting, displaying, and other operations repeatedly until the **Exit** option is selected.

Error table:

Sno	Error obtained	Rectification
1	Forgetting to update the prev pointer	Update both prev and next pointers whenever a node is inserted
2	Incorrect head.prev	Maintain head.prev = None

5(C)AIM: write a python program to implement a circular linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

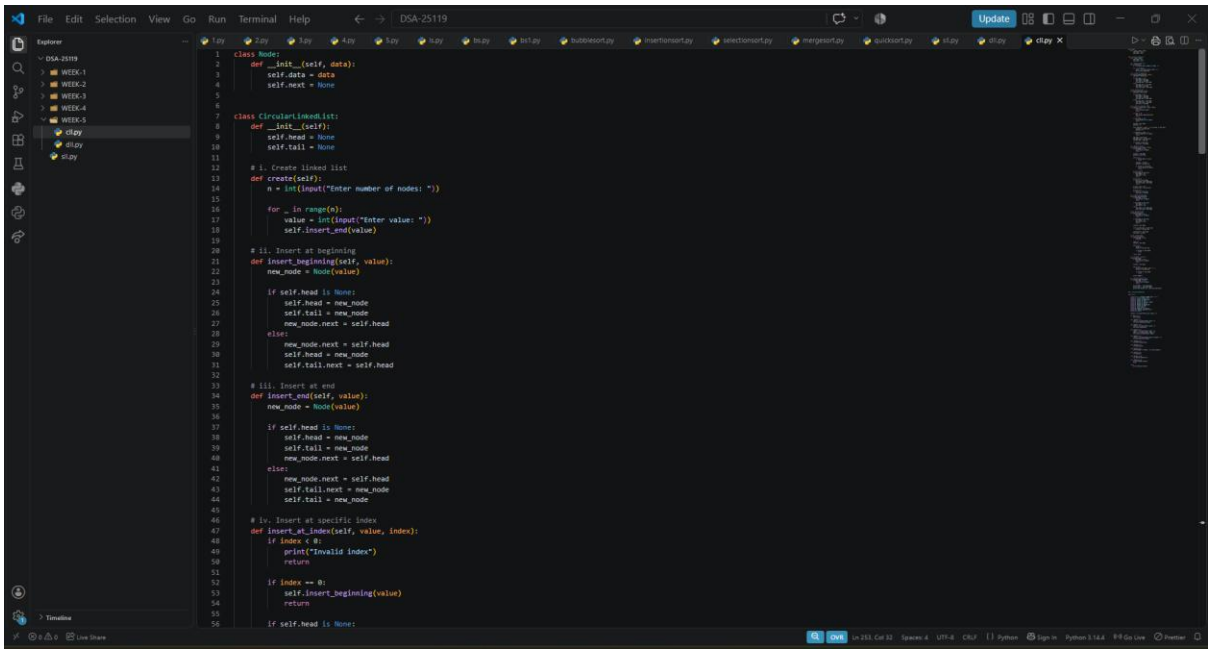
viii. count no of nodes

ix. display or travsere

x: display the head and tail

xi. exit

code:



```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for _ in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

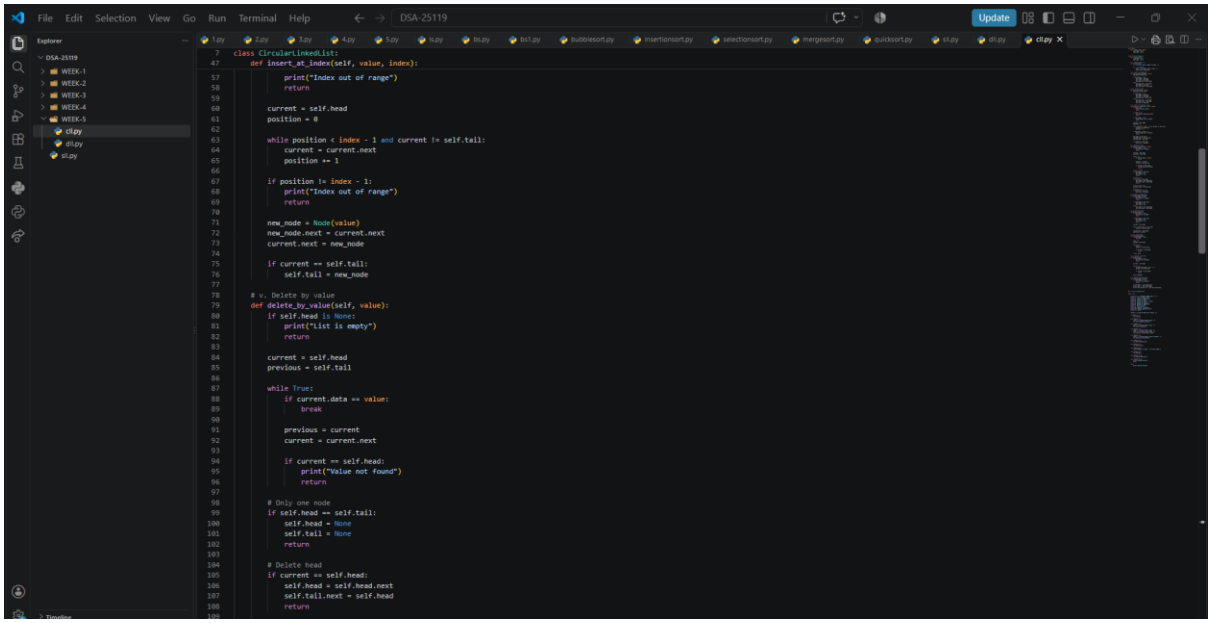
    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.tail.next = new_node
            self.tail = new_node

    # iv. Insert at specific index
    def insert_at_index(self, value, index):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_beginning(value)
            return

        if self.head is None:
```



```python
class CircularLinkedList:
    def insert_at_index(self, value, index):
            print("Index out of range")
            return

        current = self.head
        position = 0

        while position < index - 1 and current != self.tail:
            current = current.next
            position += 1

        if position != index - 1:
            print("Index out of range")
            return

        new_node = Node(value)
        new_node.next = current.next
        current.next = new_node

        if current == self.tail:
            self.tail = new_node

    # v. Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        current = self.head
        previous = self.tail

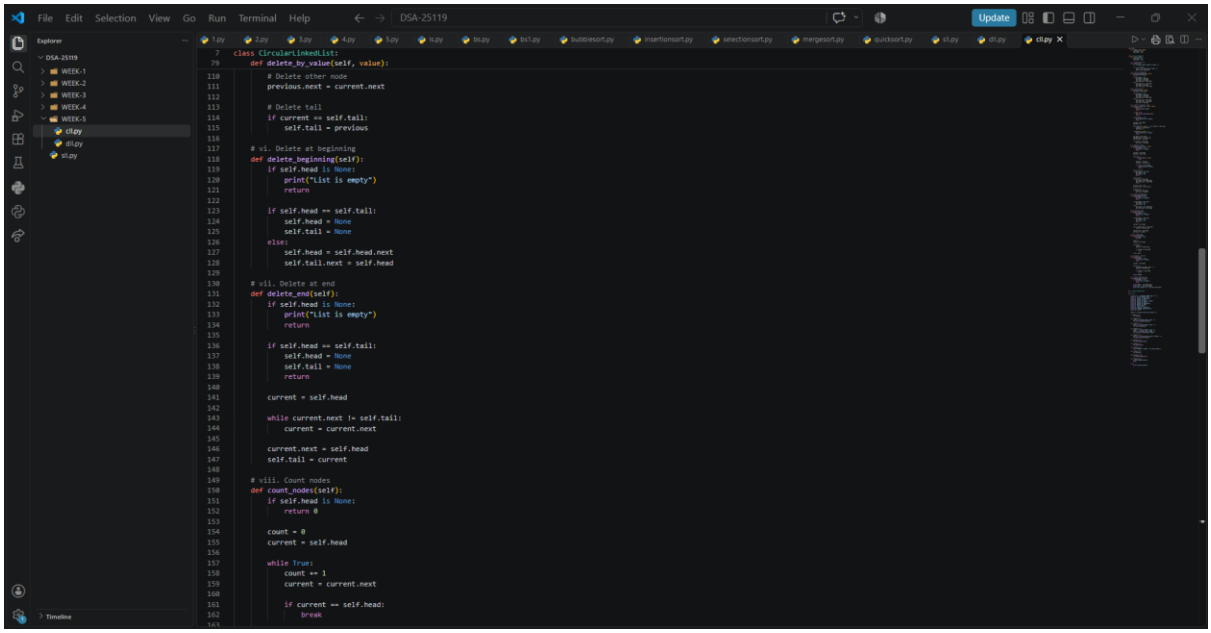
        while True:
            if current.data == value:
                break

            previous = current
            current = current.next

            if current == self.head:
                print("Value not found")
                return

        # Only one node
        if self.head == self.tail:
            self.head = None
            self.tail = None
            return

        # Delete head
        if current == self.head:
            self.head = self.head.next
            self.tail.next = self.head
            return
```



```
class CircularLinkedList:
    def delete_by_value(self, value):
        # Delete other node
        previous.next = current.next

        # Delete tail
        if current == self.tail:
            self.tail = previous

    # vi. Delete at beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head == self.tail:
            self.head = None
            self.tail = None
        else:
            self.head = self.head.next
            self.tail.next = self.head

    # vii. Delete at end
    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head == self.tail:
            self.head = None
            self.tail = None
            return

        current = self.head

        while current.next != self.tail:
            current = current.next

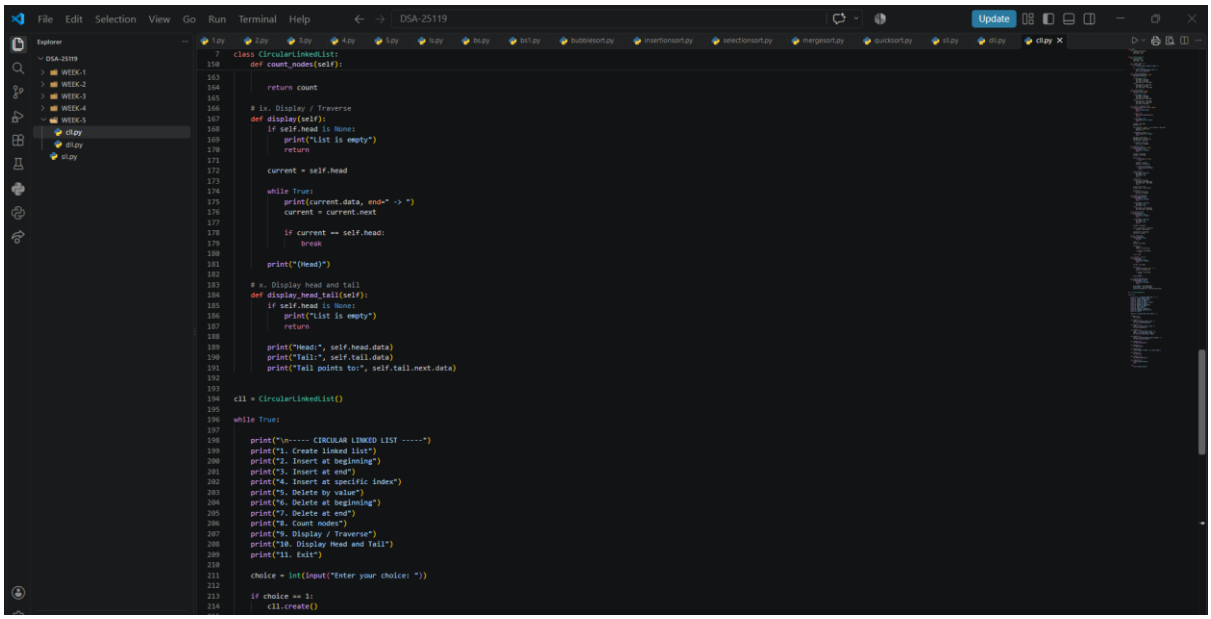
        current.next = self.head
        self.tail = current

    # viii. Count nodes
    def count_nodes(self):
        if self.head is None:
            return 0

        count = 0
        current = self.head

        while True:
            count += 1
            current = current.next

            if current == self.head:
                break
```



```
class CircularLinkedList:
    def count_nodes(self):

        return count

    # ix. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while True:
            print(current.data, end=" -> ")
            current = current.next

            if current == self.head:
                break

        print("(Head)")

    # x. Display head and tail
    def display_head_tail(self):
        if self.head is None:
            print("List is empty")
            return

        print("Head:", self.head.data)
        print("Tail:", self.tail.data)
        print("Tail points to:", self.tail.next.data)

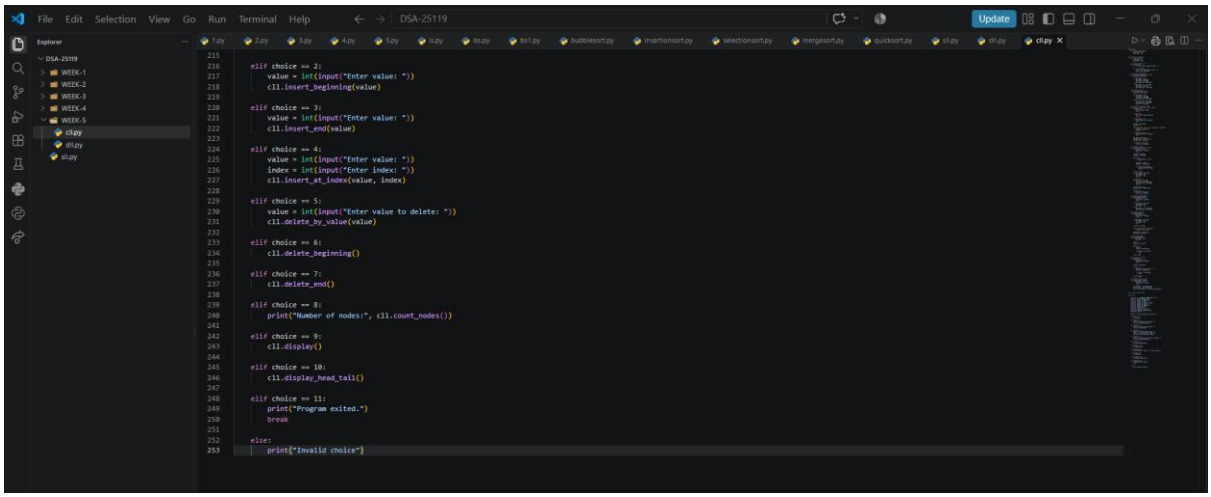

cll = CircularLinkedList()

while True:

    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Display Head and Tail")
    print("11. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()
```



```

    elif choice == 2:
        value = int(input("Enter value: "))
        cll.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        cll.insert_end(value)

    elif choice == 4:
        value = int(input("Enter value: "))
        index = int(input("Enter index: "))
        cll.insert_at_index(value, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        cll.delete_by_value(value)

    elif choice == 6:
        cll.delete_beginning()

    elif choice == 7:
        cll.delete_end()

    elif choice == 8:
        print("Number of nodes:", cll.count_nodes())

    elif choice == 9:
        cll.display()

    elif choice == 10:
        cll.display_head_tail()

    elif choice == 11:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

Explanation :

1. A **circular singly linked list** is a collection of nodes where each node contains **data** and a **link to the next node**.
2. In a circular linked list, the **last node points back to the first node (head)** instead of pointing to None.
3. The program uses two references:

- **Head** – points to the first node.
- **Tail** – points to the last node.

4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. Each new node is inserted at the end of the list.
7. If the list is empty, the first node becomes both the **head and tail**, and its next points back to itself.
8. During **insertion at the beginning**, the new node is placed before the current head. The new node becomes the head, and the tail is connected to the new head.
9. During **insertion at the end**, the new node is added after the current tail. The new node becomes the tail, and its next points to the head.
10. During **insertion at a specific index**, the list is traversed until the required position is reached, and the new node is inserted between two existing nodes.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by connecting its previous node directly to its next node.
12. If the node to be deleted is the **head**, the next node becomes the new head and the tail is connected to it.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is connected to the head.
14. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
15. During **deletion at the end**, the node before the tail is found and made the new tail.
16. The **count operation** traverses the list and counts all the nodes. Traversal stops when the head is reached again.

17. The **display/traverse operation** starts from the head and visits every node until it reaches the head again.
18. Since the list is circular, traversal does not use current != None. Instead, it stops when current becomes equal to head.
19. The program also displays the **head, tail, and tail.next** to show the circular connection.
20. The program is **menu-driven**, so the user can repeatedly perform different operations until the **Exit** option is selected.
21. The most important property maintained throughout the program is:

tail.next = head

22. Thus, the structure of the circular linked list is:

HEAD → Node → Node → Node → TAIL

↑ |

|_____|

This ensures that the linked list remains **circular after every insertion and deletion operation**.

Error table:

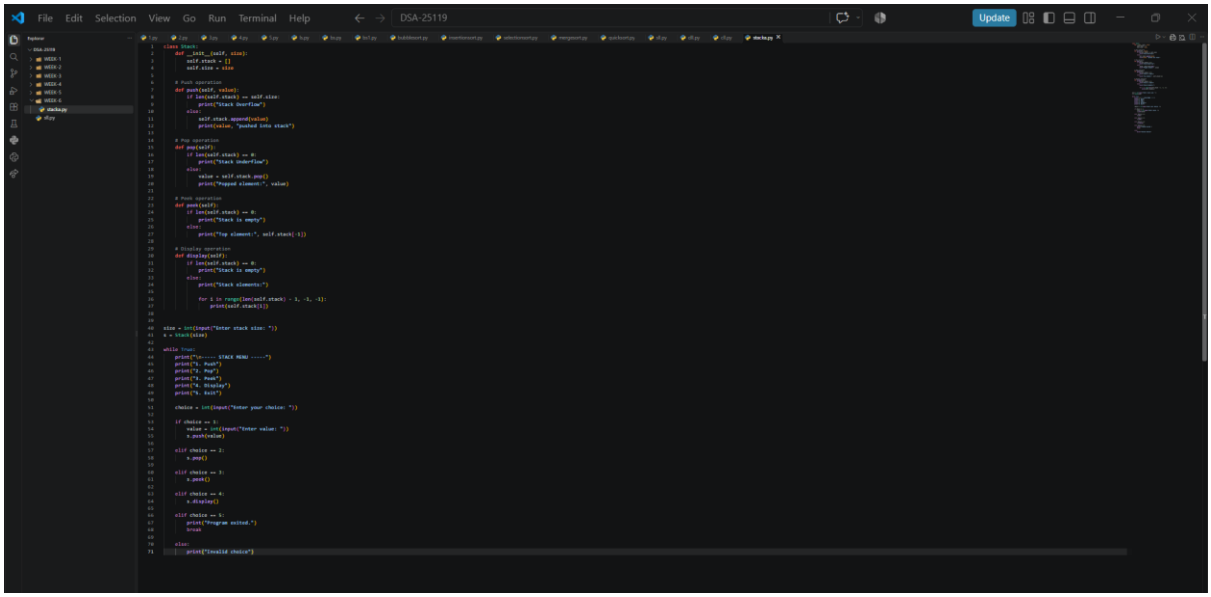
sno	Error obtained	Rectification
1	Using temp != None as the traversal condition	Stop traversal when temp == head
2	Forgetting to update tail.next when the head changes	After changing the head, update tail.next = head

WEEK-6

6A.

Aim: write a python program to implement the stack using the arrays. And also implement stack operations such as push, pop, peek and also display the stack elements.

Code:



```
class Stack:
    def __init__(self, size):
        self.stack = []
        self.size = size

    # Push operation
    def push(self, value):
        if len(self.stack) == self.size:
            print("Stack Overflow")
        else:
            self.stack.append(value)
            print(value, "pushed into stack")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow")
        else:
            value = self.stack.pop()
            print("Popped element:", value)

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[-1])

    # Display operation
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])

size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        s.push(value)

    elif choice == 2:
        s.pop()

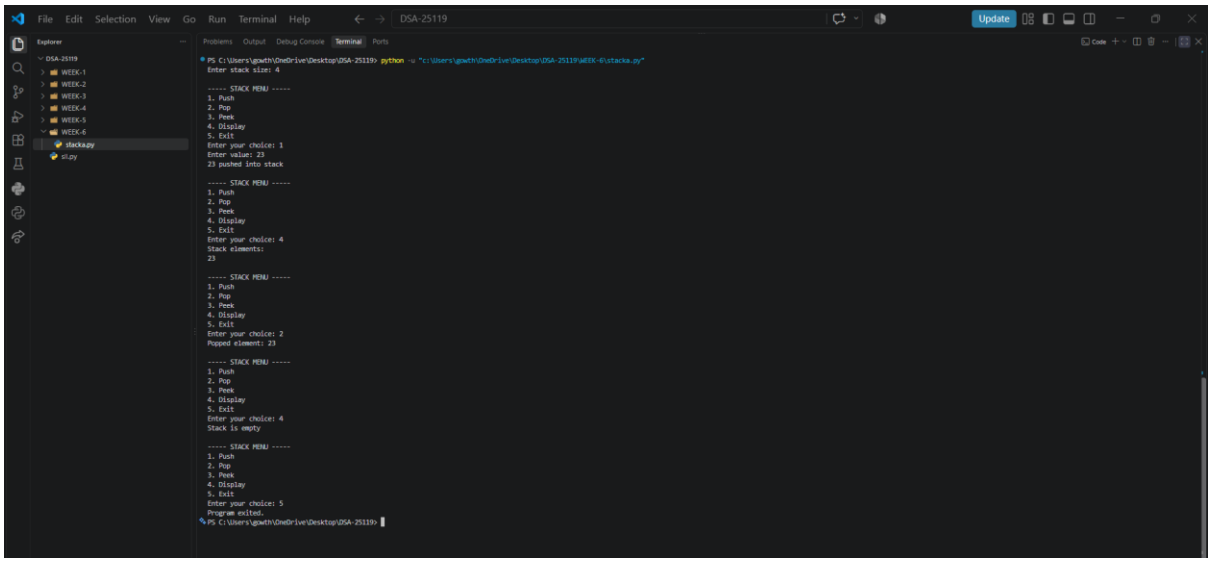
    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



```
Enter stack size: 4

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 23
23 pushed into stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
23

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 23

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack is empty

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
```

Explanation:

1. Class and Initialization

- A class named Stack is created.
- The __init__() method acts as the constructor.

- An empty list self.stack is created to store stack elements.
- The stack follows the **LIFO (Last In, First Out)** principle.

2. Push Operation

- The push() method is used to insert an element into the stack.
- The append() function adds the element to the end of the list.
- The newly inserted element becomes the **top element**.
- A message is displayed after the element is inserted.

3. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether the stack is empty.
- If the stack is empty, **Stack Underflow** is displayed.
- Otherwise, the pop() function removes the last element.
- The removed element is displayed.

4. Peek Operation

- The peek() method is used to view the top element.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.stack[-1] is used to access the top element.
- The top element is displayed without removing it.

5. Display Operation

- The display() method is used to display all stack elements.
- The program first checks whether the stack is empty.
- If empty, **Stack is empty** is displayed.
- Otherwise, a for loop is used to traverse the stack.
- The elements are displayed from **top to bottom**.

6. Count Operation

- The count() method is used to find the number of elements in the stack.
- The len() function returns the number of elements.
- The count is displayed to the user.

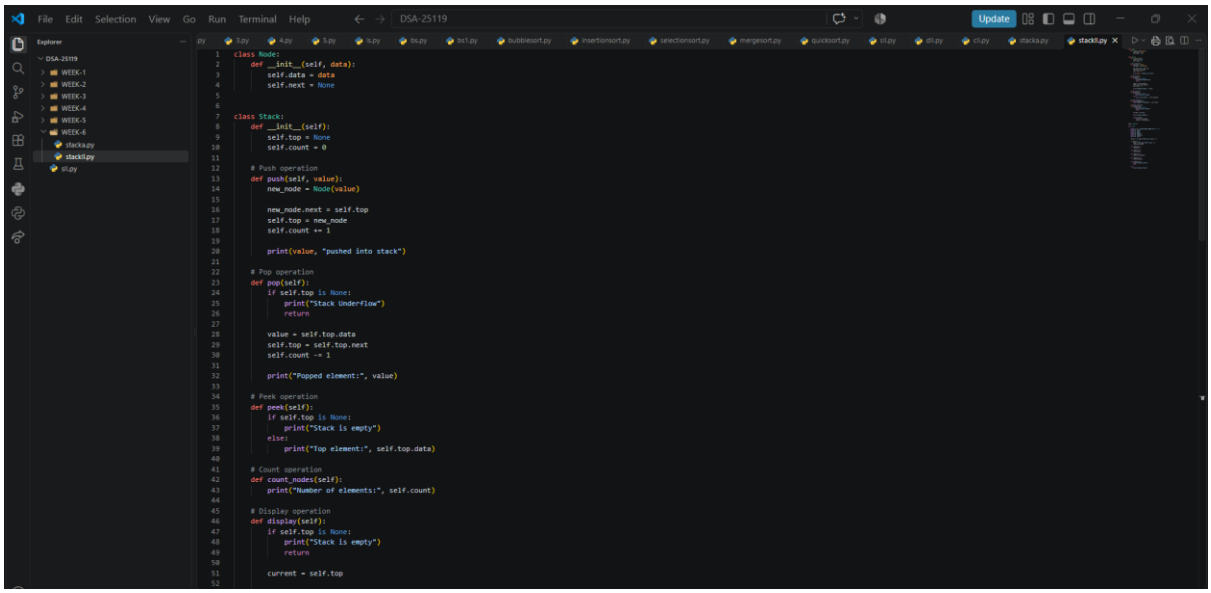
Error table:

sno	Error obtained	Error rectification
1	Stack underflow	Check whether the stack is empty before poping
2	Name error	Define the class before creating the object

5b.

Aim: write a python program to implement the stack using the linked list. And also implement stack operations such as push, pop, peek,count and also display the stack elements.

Code:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None
        self.count = 0

    # Push operation
    def push(self, value):
        new_node = Node(value)

        new_node.next = self.top
        self.top = new_node
        self.count += 1

        print(value, "pushed into stack")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
            return

        value = self.top.data
        self.top = self.top.next
        self.count -= 1

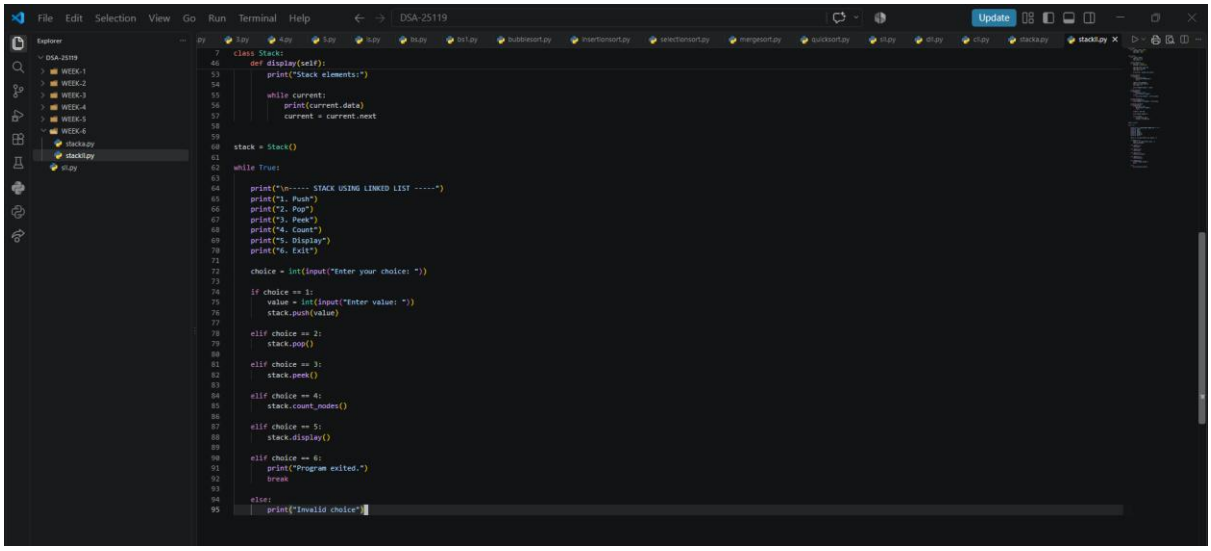
        print("Popped element:", value)

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Count operation
    def count_nodes(self):
        print("Number of elements:", self.count)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
            return

        current = self.top
```



```
class Stack:
    def display(self):
        print("Stack elements:")

        while current:
            print(current.data)
            current = current.next

stack = Stack()

while True:

    print("\n----- STACK USING LINKED LIST -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Count")
    print("5. Display")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        stack.push(value)

    elif choice == 2:
        stack.pop()

    elif choice == 3:
        stack.peek()

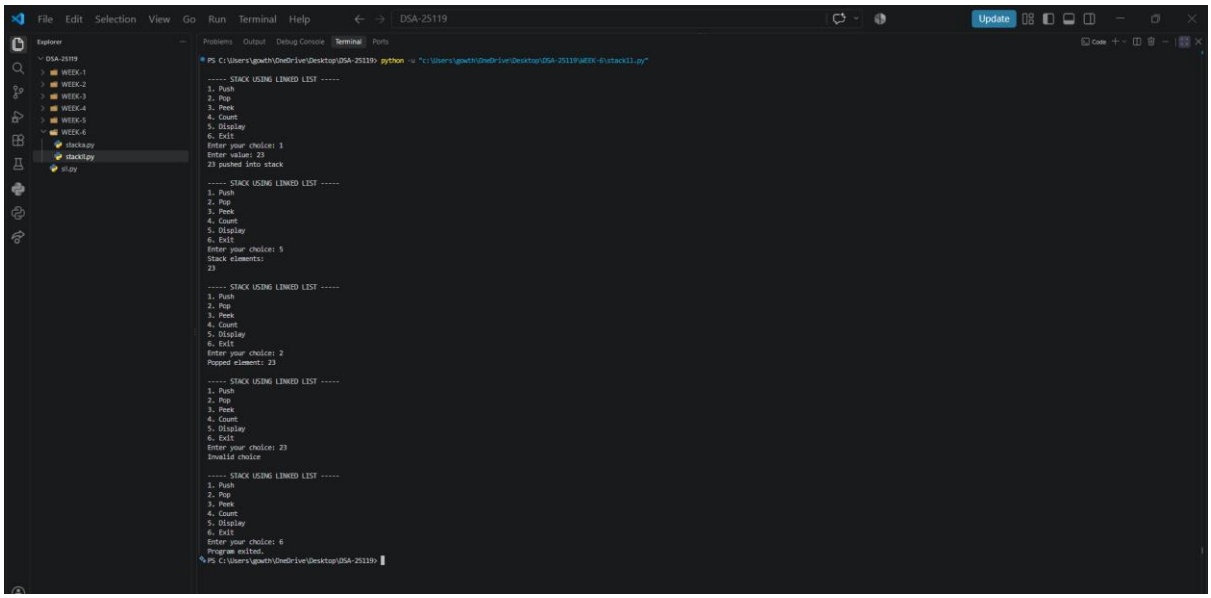
    elif choice == 4:
        stack.count_nodes()

    elif choice == 5:
        stack.display()

    elif choice == 6:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-6\stack11.py"

----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 1
Enter value: 23
23 pushed into stack

----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 5
Stack elements:
23

----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 2
Popped element: 23

----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 23
Invalid choice

----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 6
Program exited.
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

Explanation:

1. Node Class

- A class named Node is created to represent each element of the linked list.
- The __init__() method initializes a node.
- self.data stores the value of the node.
- self.next stores the reference to the next node.
- Initially, self.next is set to None.

2. Stack Class

- A class named Stack is created to implement the stack operations.
- The __init__() method initializes the stack.

- self.top = None indicates that the stack is initially empty.
- The top variable stores the reference to the top node of the stack.

3. Push Operation

- The push() method is used to insert an element into the stack.
- A new node is created using Node(data).
- The new node is connected to the current top node using:

new_node.next = self.top

- The new node is then made the top node using:

self.top = new_node

- The inserted element is displayed.
- The operation follows the **LIFO (Last In, First Out)** principle.

4. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether self.top is None.
- If self.top is None, the stack is empty and **Stack Underflow** is displayed.
- Otherwise, the data of the top node is stored in data.
- The top is moved to the next node using:

self.top = self.top.next

- The removed element is displayed.

5. Peek Operation

- The peek() method is used to view the top element without removing it.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.top.data is used to display the top element.
- The stack remains unchanged after the peek operation.

6. Display Operation

- The display() method is used to display all the elements of the stack.
- The program first checks whether the stack is empty.
- A temporary variable current is initialized with self.top.

- A while loop is used to traverse the linked list.
- current.data displays the value of each node.
- current = current.next moves to the next node.
- The elements are displayed from **top to bottom**.

7. Count Operation

- The count() method is used to find the number of elements in the stack.
- A variable count is initialized to 0.
- A temporary variable current starts from self.top.
- The linked list is traversed using a while loop.
- The count value is increased for every node.
- The total number of elements is displayed.

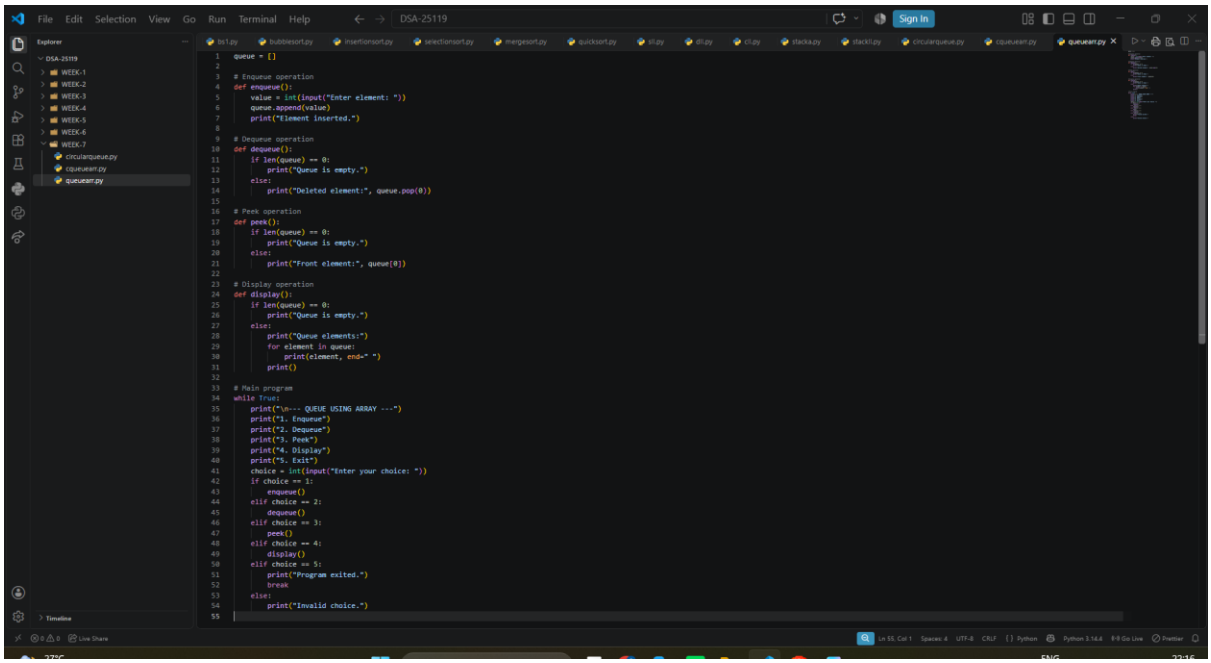
Error table:

sno	Error obtained	Error rectification
1	Incorrect push	Set new_node.next = self.top before updating self.top
2	Infinte loop	Use break when choice == 6

Week-7

Aim : Write a python program to implement a queue using an Array, Linked list, circular queue. Implement the queue operations such as enqueue, dequeue, peak and also display the queue elements.

A.AIM: Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using array



```
queue = []

# Enqueue operation
def enqueue():
    value = int(input("Enter element: "))
    queue.append(value)
    print("Element inserted.")

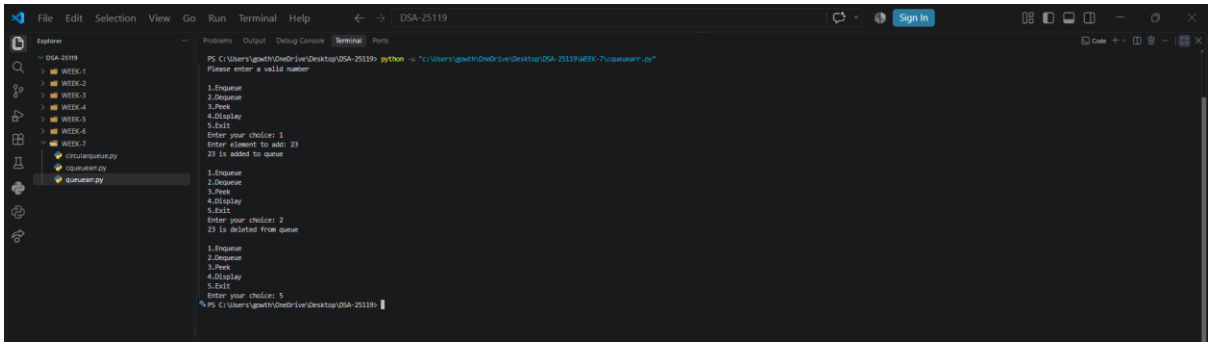
# Dequeue operation
def dequeue():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Deleted element:", queue.pop(0))

# Peek operation
def peek():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Front element:", queue[0])

# Display operation
def display():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Queue elements:")
        for element in queue:
            print(element, end=" ")
        print()

# Main program
while True:
    print("\n--- QUEUE USING ARRAY ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program exited.")
        break
    else:
        print("Invalid choice.")
```

OUTPUT:



```
Please enter a valid number

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add: 23
23 is added to queue

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
23 is deleted from queue

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
```

EXPLANATION:

- The queue list is used to store the **queue elements**.
- The enqueue() function adds an element at the **rear** of the queue.
- The dequeue() function removes an element from the **front** of the queue.
- The peek() function displays the **front element** without removing it.
- The display() function prints all the elements in the queue.
- len(queue) == 0 checks whether the queue is **empty**.

- The main program provides a **menu** to select different queue operations.
- Queue follows the **FIFO (First In, First Out)** principle.

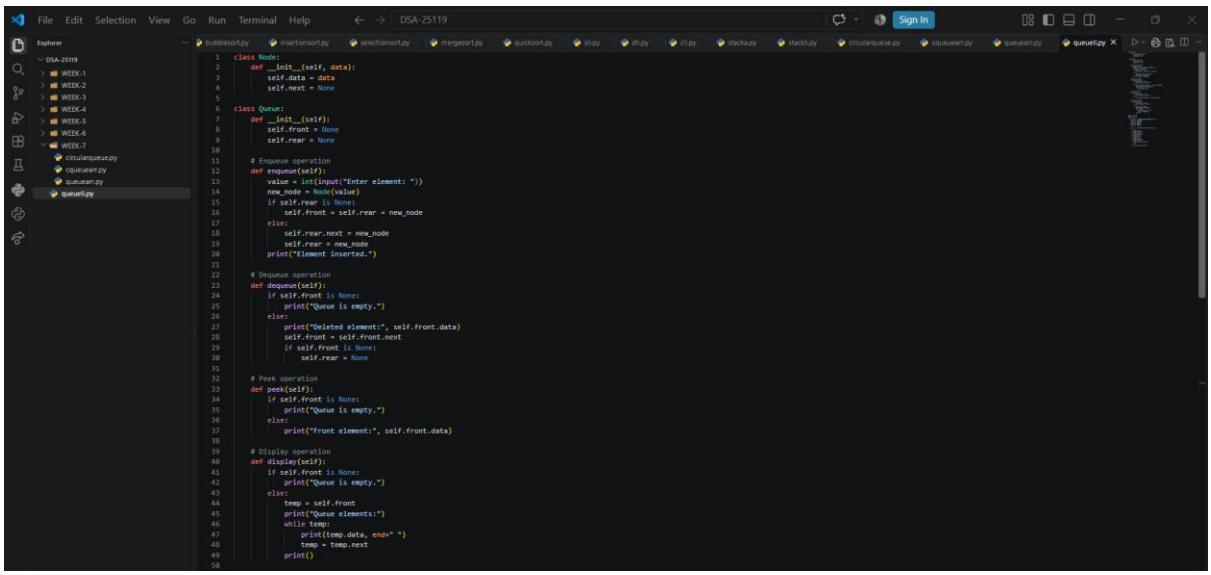
ERROR TABLE:

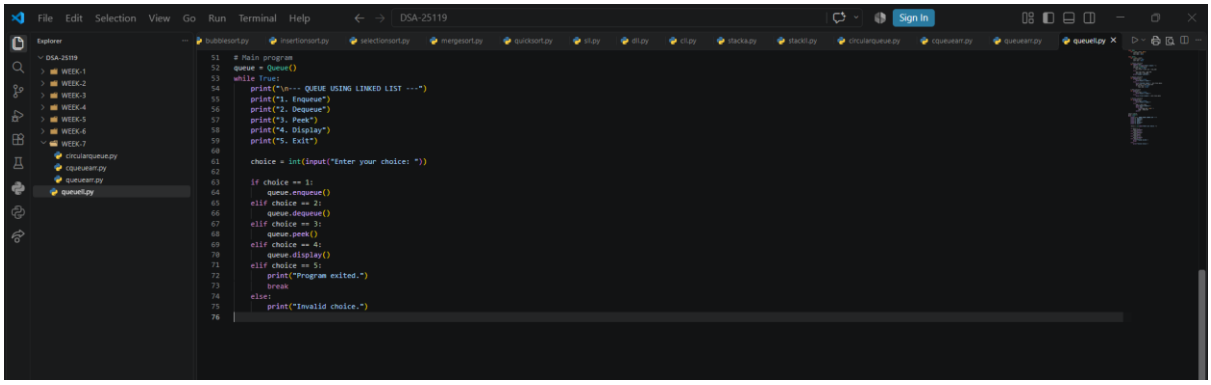
S.no.	Errors	Rectification
1.	**IndentationError** occurred because the statements inside the functions were not properly aligned.	Corrected the indentation of all code blocks.
2.	**ValueError** occurred when a noninteger value was entered.	Entered only valid integer values.
3.	**NameError** occurred because queue or a function name was misspelled.	**NameError** occurred because queue or a function name was misspelled.
4.	**IndexError** occurred when trying to access an element from an empty queue.	Checked whether the queue is empty before accessing elements.

B.AIM: Write a Python program to implement a queue and its operations such as

Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Linked list data.

CODE:



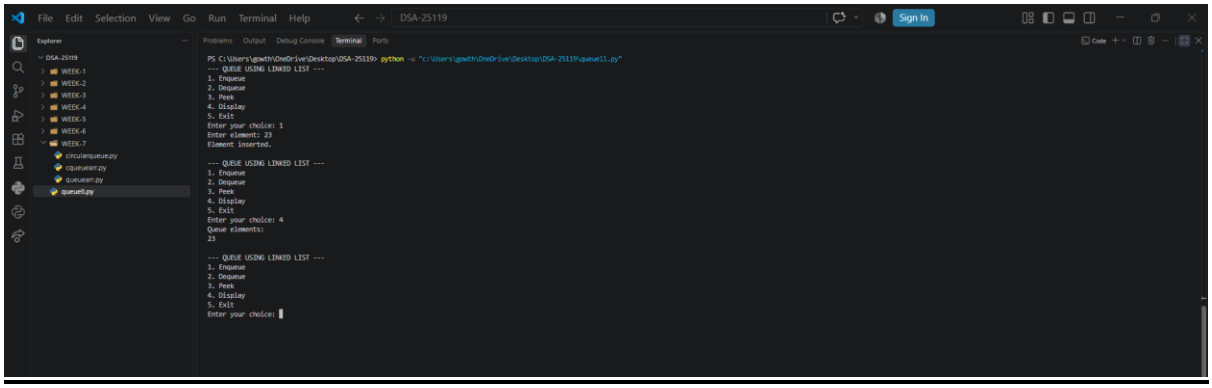


```
# Main program
queue = Queue()
while True:
    print("\n--- QUEUE USING LINKED LIST ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        queue.enqueue()
    elif choice == 2:
        queue.dequeue()
    elif choice == 3:
        queue.peek()
    elif choice == 4:
        queue.display()
    elif choice == 5:
        print("Program exited.")
        break
    else:
        print("Invalid choice.")
```

OUTPUT:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\queue11.py"
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 23
Element inserted.

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
23

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

- The Node class creates a node containing **data and a link to the next node**.
- The Queue class uses front and rear to maintain the queue.
- The enqueue() function adds a new node at the **rear**.
- The dequeue() function removes a node from the **front**.
- The peek() function displays the **front element** without removing it.
- The display() function traverses the linked list and displays all elements.
- The queue follows the **FIFO (First In, First Out)** principle.
- The main program provides a **menu-driven approach** for performing the operations.

ERROR TABLE:

S.no.	Errors	Rectification

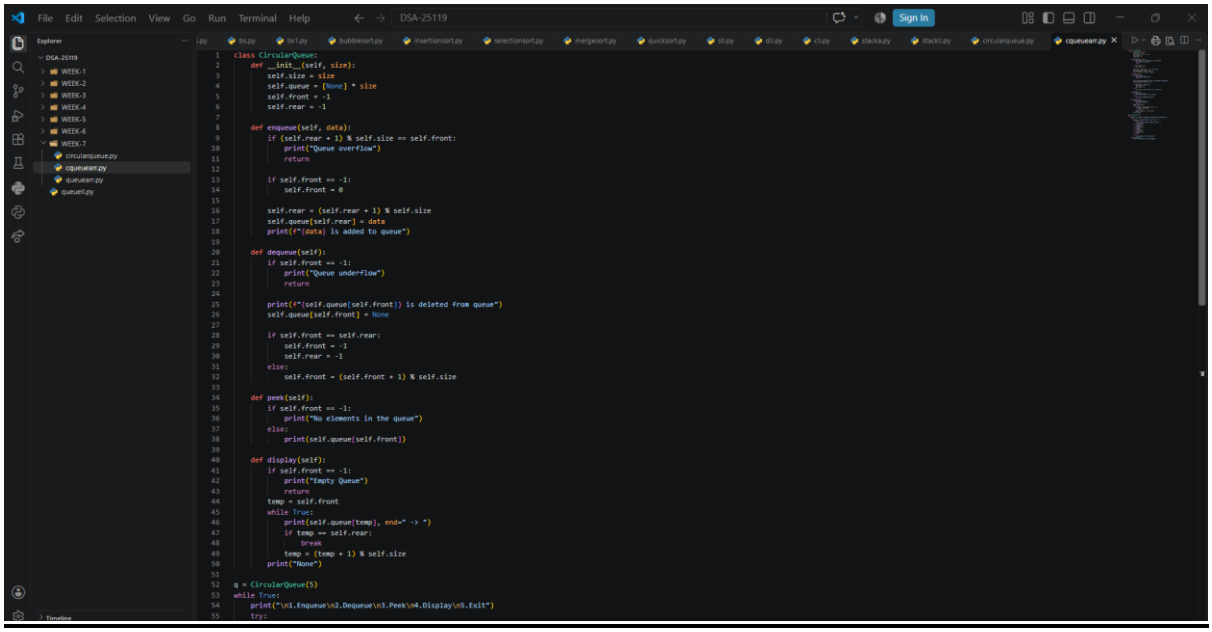
1.	**NameError** occurred because front, rear, or new_node was misspelled.	Corrected the variable names and their spelling.
2.	**AttributeError** occurred when next was accessed incorrectly.	Used self.front, self.rear, and temp.next correctly.
3.	**ValueError** occurred when a noninteger value was entered.	Entered only valid integer values.
4.	**Empty queue error** occurred when Dequeue or Peek was performed without elements.	Checked self.front is None before performing the operation.
5.	**SyntaxError** occurred due to incorrect Python syntax.	Checked and corrected the syntax of the program.

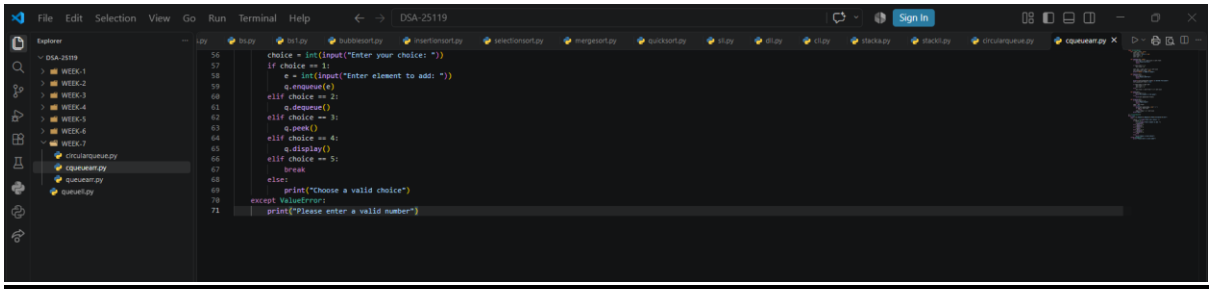
C.AIM:

Write a Python program to implement a queue and its operations such as

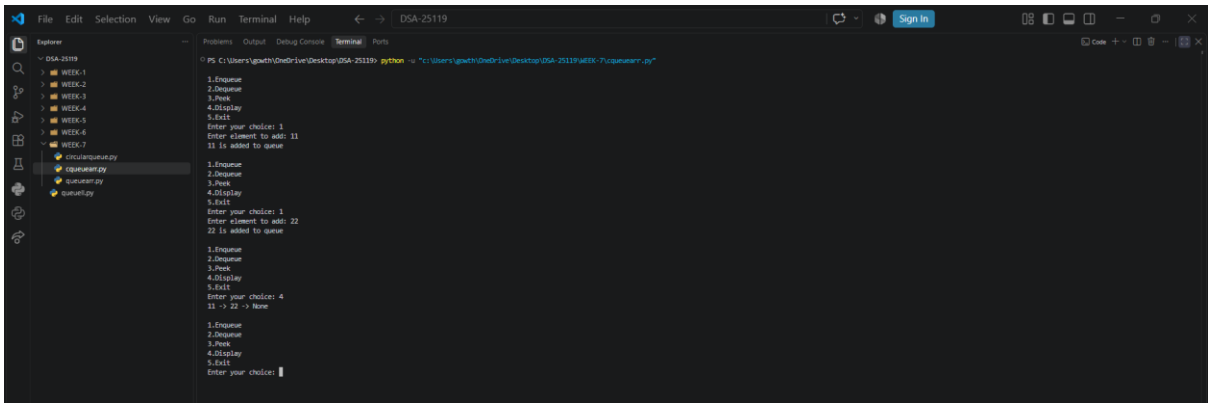
Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue array structures

CODE:





OUTPUT:



EXPLANATION:

- The CircularQueue class creates a queue using an array.
- front represents the position of the first element, while rear represents the last element.
- enqueue() adds a new element at the rear.
- dequeue() removes an element from the front.
- peek() displays the front element without deleting it.
- display() prints all the elements from front to rear.
- The % size operation makes the rear and front wrap around the array.
- The main program provides a menu-driven interface.
- The queue follows the FIFO (First In, First Out) principle.

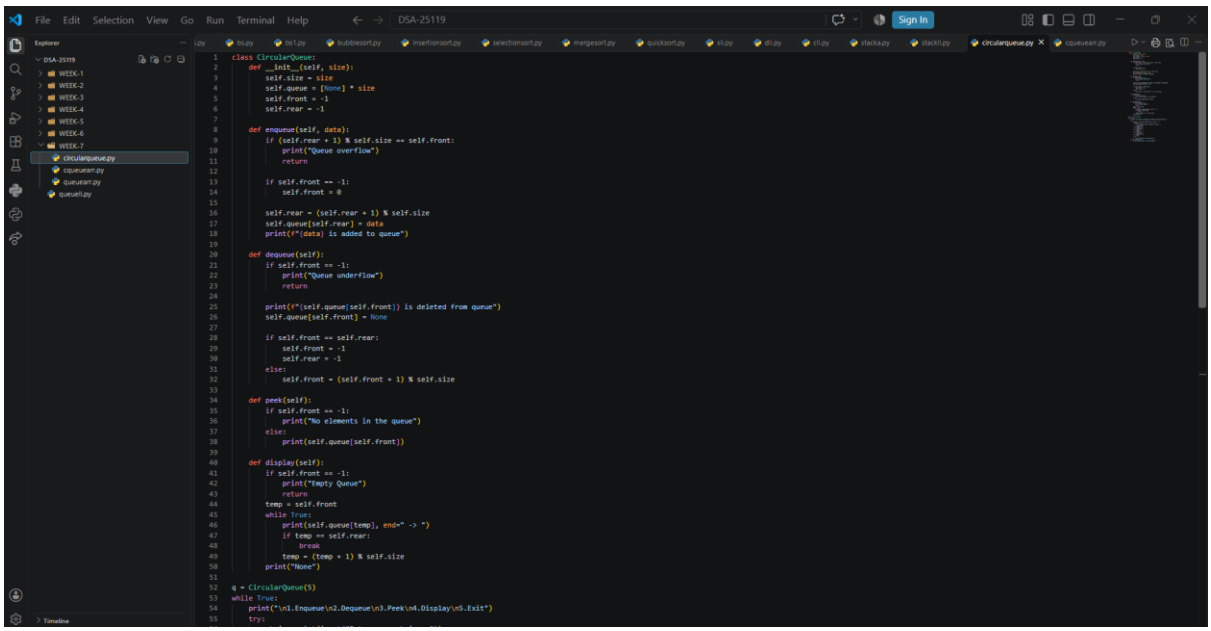
ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError occurred because the statements were not properly aligned.	Corrected the indentation of all code blocks.
2.	ValueError occurred when a noninteger value was entered.	Entered only valid integer values.
3.	NameError occurred because front, rear, or queue was misspelled.	Corrected the variable names and spelling.
4.	IndexError occurred when an invalid array position was accessed.	Used proper circular indexing with % size.
5.	Queue full error occurred when an element was inserted into a full queue.	Checked the full condition before enqueueing.

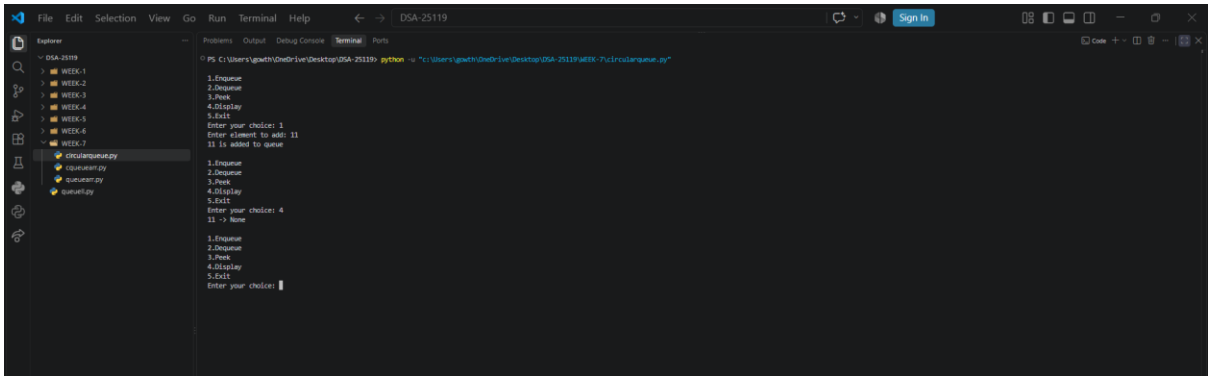
D.AIM: Write a Python program to implement a queue and its operations such as

Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue Linked list structures.

CODE:



OUTPUT:



EXPLANATION:

- The Node class creates a node containing **data and a next link**.
- The CircularQueue class maintains the queue using front and rear.
- The enqueue() function adds a node at the **rear** and connects it back to the front.
- The dequeue() function removes the **front node**.
- The peek() function displays the **front element** without removing it.
- The display() function traverses the circular linked list until it reaches the **front again**.
- front and rear are set to None when the queue becomes empty.
- The main program provides a **menu-driven approach**.
- The circular queue follows the **FIFO (First In, First Out)** principle.
-

ERROR TABLE:

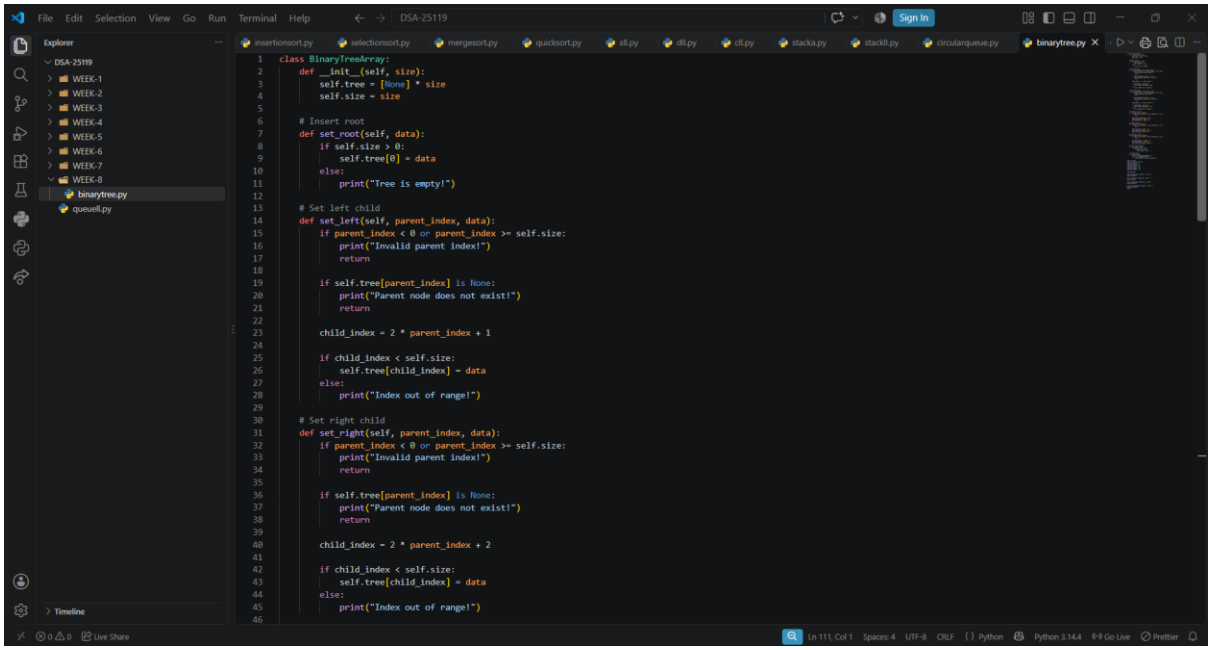
S.no.	Errors	Rectification
1.	**NameError** occurred because front, rear, or new_node was misspelled.	Corrected the variable names and spelling.
2.	**AttributeError** occurred when the next link was not accessed correctly.	Used self.front, self.rear, and temp.next correctly.

3.	**ValueError** occurred when a noninteger value was entered.	Entered only valid integer values.
4.	**Empty queue error** occurred when Dequeue or Peek was performed on an empty queue.	Checked self.front is None before performing the operation.
5.	**Circular link error** occurred when the rear node was not connected back to the front node.	Updated self.rear.next = self.front after insertion and deletion.

WEEK-8

A.AIM: Write a Python program to implement a binary tree with its operations using an arrays

CODE:



```python
class BinaryTreeArray:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    # Insert root
    def set_root(self, data):
        if self.size > 0:
            self.tree[0] = data
        else:
            print("Tree is empty!")

    # Set left child
    def set_left(self, parent_index, data):
        if parent_index < 0 or parent_index >= self.size:
            print("Invalid parent index!")
            return

        if self.tree[parent_index] is None:
            print("Parent node does not exist!")
            return

        child_index = 2 * parent_index + 1

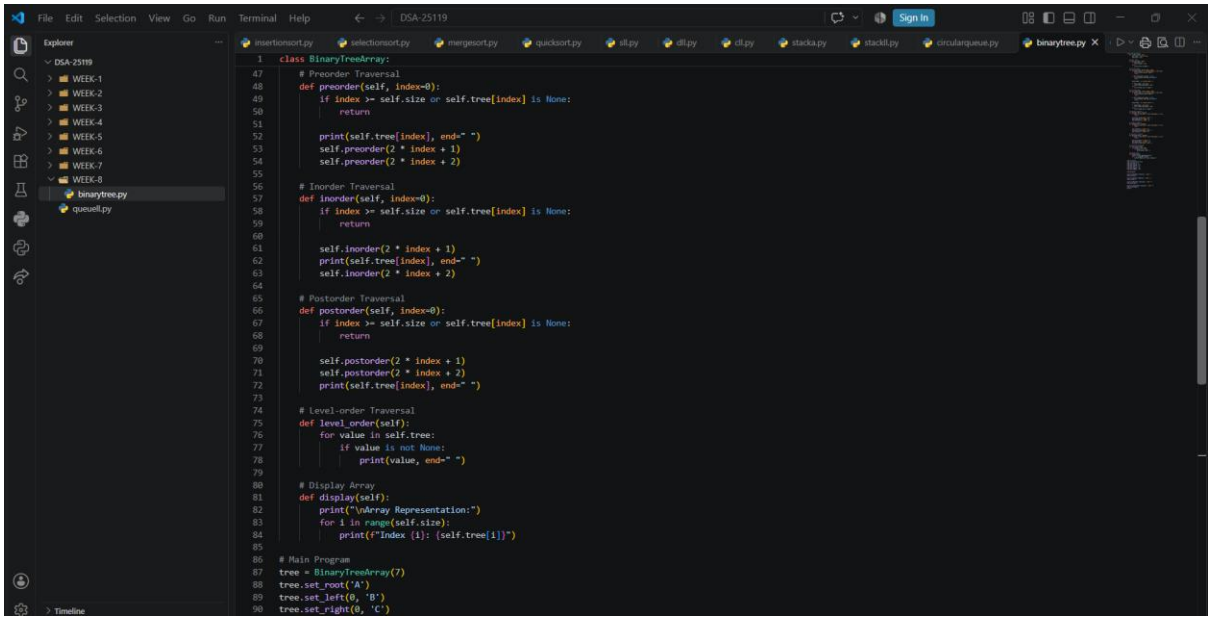
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")

    # Set right child
    def set_right(self, parent_index, data):
        if parent_index < 0 or parent_index >= self.size:
            print("Invalid parent index!")
            return

        if self.tree[parent_index] is None:
            print("Parent node does not exist!")
            return

        child_index = 2 * parent_index + 2

        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")
```



```python
class BinaryTreeArray:
    # Preorder Traversal
    def preorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return

        print(self.tree[index], end=" ")
        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)

    # Inorder Traversal
    def inorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return

        self.inorder(2 * index + 1)
        print(self.tree[index], end=" ")
        self.inorder(2 * index + 2)

    # Postorder Traversal
    def postorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return

        self.postorder(2 * index + 1)
        self.postorder(2 * index + 2)
        print(self.tree[index], end=" ")

    # Level-order Traversal
    def level_order(self):
        for value in self.tree:
            if value is not None:
                print(value, end=" ")

    # Display Array
    def display(self):
        print("\nArray Representation:")
        for i in range(self.size):
            print(f"Index {i}: {self.tree[i]}")

# Main Program
tree = BinaryTreeArray(7)
tree.set_root('A')
tree.set_left(0, 'B')
tree.set_right(0, 'C')
```



```
tree.set_left(1, 'D')
tree.set_right(1, 'E')
tree.set_right(2, 'F')

tree.display()

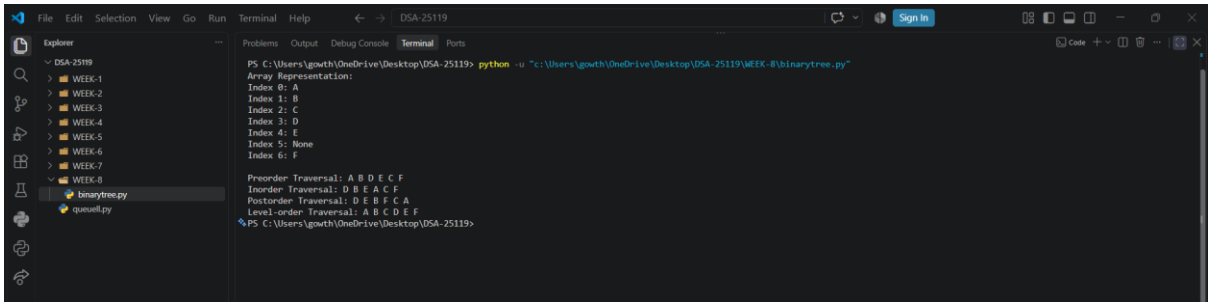
print("\nPreorder Traversal:", end=" ")
tree.preorder()

print("\nInorder Traversal:", end=" ")
tree.inorder()

print("\nPostorder Traversal:", end=" ")
tree.postorder()

print("\nLevel-order Traversal:", end=" ")
tree.level_order()
print()
```

OUTPUT:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-8\binarytree.py"
Array Representation:
Index 0: A
Index 1: B
Index 2: C
Index 3: D
Index 4: E
Index 5: None
Index 6: F

Preorder Traversal: A B D E C F
Inorder Traversal: D B E A C F
Postorder Traversal: D E B F C A
Level-order Traversal: A B C D E F
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

EXPLANATION:

- The BinaryTreeArray class represents a binary tree using an array.
- The __init__() function initializes an array of the given size with None values.
- The set_root() function inserts the root node at index 0.
- The set_left() function inserts the left child of a parent node using the formula 2 * parent_index + 1.
- The set_right() function inserts the right child of a parent node using the formula 2 * parent_index + 2.
- The preorder() function traverses the tree in the order Root → Left → Right.
- The inorder() function traverses the tree in the order Left → Root → Right.
- The postorder() function traverses the tree in the order Left → Right → Root.
- The level_order() function displays the nodes level by level by traversing the array.
- The display() function displays each array index and its corresponding node value.
- The main program creates a binary tree of size 7 and inserts nodes A, B, C, D, E, F.
- The binary tree follows the parent-child relationship, where each node can have at most two children: a left child and a right child.

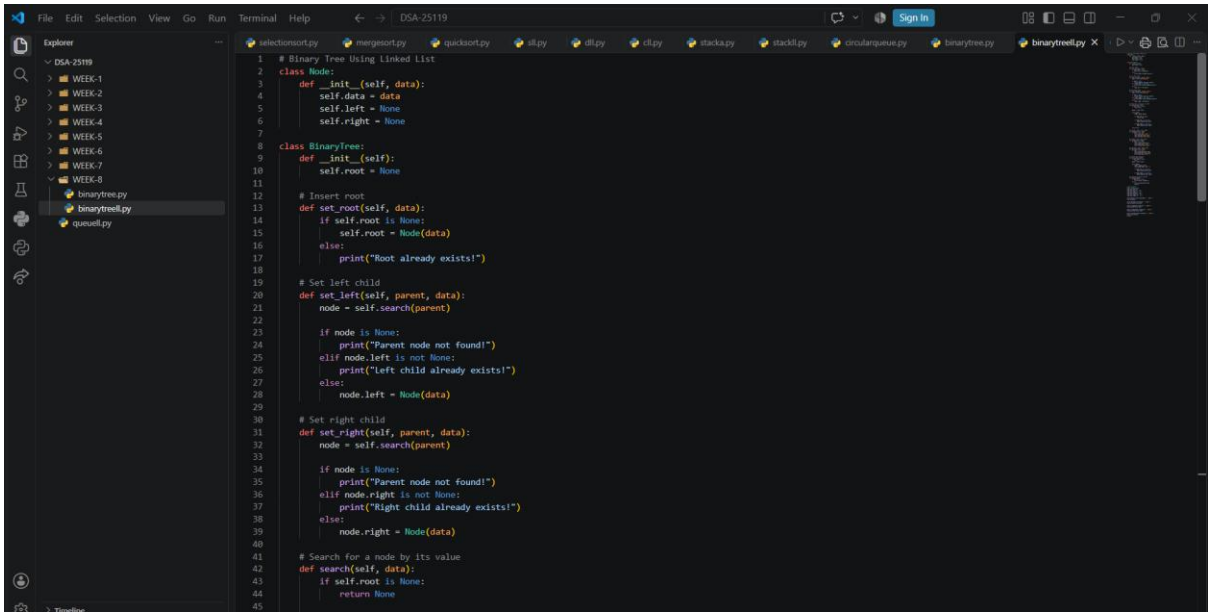
- The program demonstrates array representation and different tree traversal techniques.

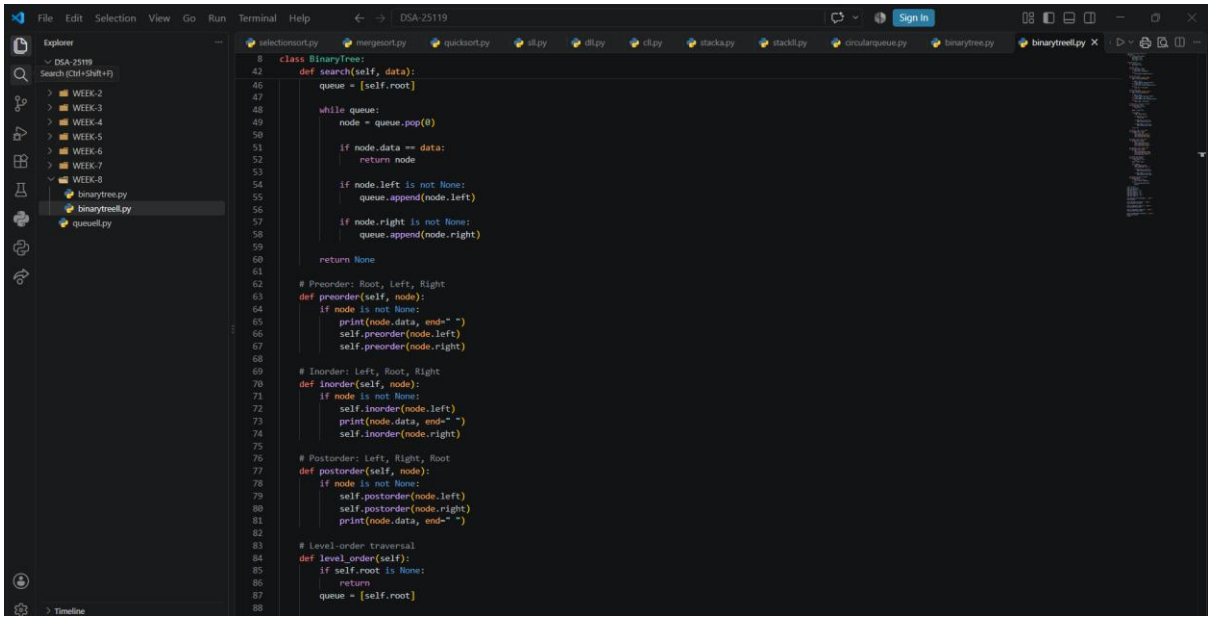
ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndexError** may occur if an invalid index is accessed in the tree array.	Checked the index range before accessing the array.
2.	**Incorrect child index** occurred when the left or right child position was calculated incorrectly.	Used 2 * parent_index + 1 for the left child and 2 * parent_index + 2 for the right child.
3.	**Parent node error** occurred when trying to insert a child for a parent that does not exist.	Checked whether self.tree[parent_index] is None before insertion.
4.	Index out of range error occurred when the child index exceeded the array size.	Checked whether child_index < self.size before inserting the child.
5.	Incorrect traversal output occurred when the preorder, inorder, or postorder sequence was not followed correctly.	Used the correct traversal orders: Root–Left–Right, Left–Root–Right, and Left–Right–Root.

B.AIM: Write a Python program to implement a binary tree with its operations using an Linked list

CODE:





```python
class BinaryTree:
    def search(self, data):
        queue = [self.root]

        while queue:
            node = queue.pop(0)

            if node.data == data:
                return node

            if node.left is not None:
                queue.append(node.left)

            if node.right is not None:
                queue.append(node.right)

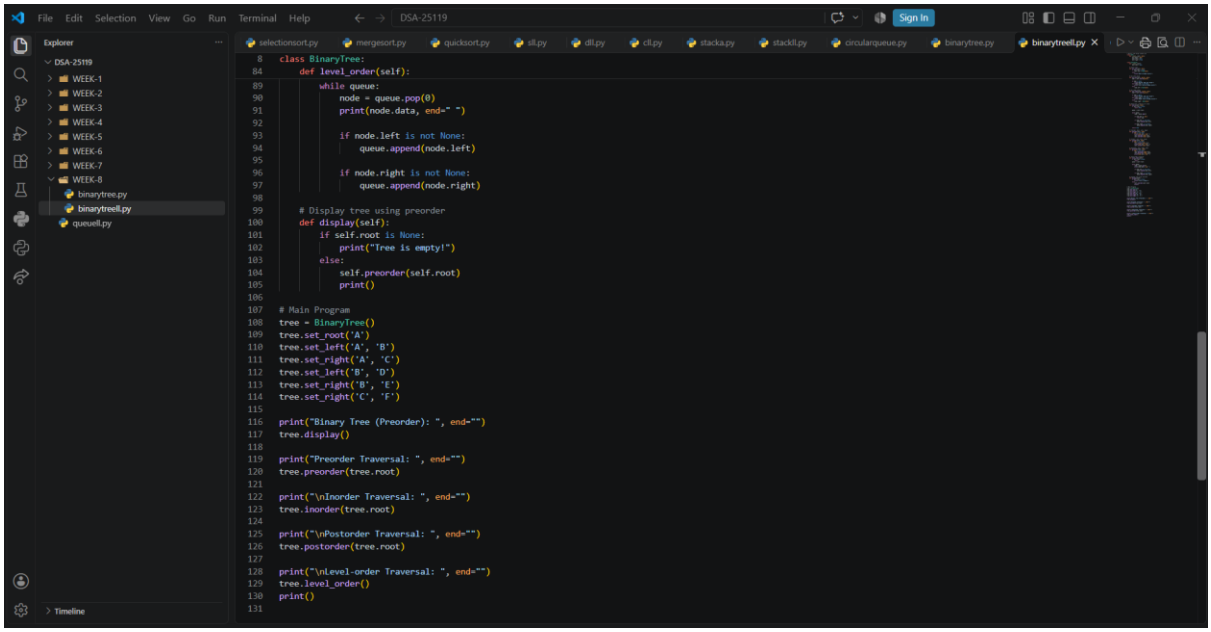
        return None

    # Preorder: Root, Left, Right
    def preorder(self, node):
        if node is not None:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    # Inorder: Left, Root, Right
    def inorder(self, node):
        if node is not None:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    # Postorder: Left, Right, Root
    def postorder(self, node):
        if node is not None:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    # Level-order traversal
    def level_order(self):
        if self.root is None:
            return
        queue = [self.root]
```



```python
class BinaryTree:
    def level_order(self):
        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")

            if node.left is not None:
                queue.append(node.left)

            if node.right is not None:
                queue.append(node.right)

    # Display tree using preorder
    def display(self):
        if self.root is None:
            print("Tree is empty!")
        else:
            self.preorder(self.root)
            print()

# Main Program
tree = BinaryTree()
tree.set_root('A')
tree.set_left('A', 'B')
tree.set_right('A', 'C')
tree.set_left('B', 'D')
tree.set_right('B', 'E')
tree.set_right('C', 'F')

print("Binary Tree (Preorder): ", end="")
tree.display()

print("Preorder Traversal: ", end="")
tree.preorder(tree.root)

print("\nInorder Traversal: ", end="")
tree.inorder(tree.root)

print("\nPostorder Traversal: ", end="")
tree.postorder(tree.root)

print("\nLevel-order Traversal: ", end="")
tree.level_order()
print()
```

OUTPUT:



```
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119> python -u "c:\Users\gowth\OneDrive\Desktop\DSA-25119\WEEK-8\binarytreell.py"
Binary Tree (Preorder): A B D E C F
Preorder Traversal: A B D E C F
Inorder Traversal: D B E A C F
Postorder Traversal: D E B F C A
Level-order Traversal: A B C D E F
PS C:\Users\gowth\OneDrive\Desktop\DSA-25119>
```

EXPLANATION:

- The Node class creates a node containing data, left, and right links.
- The BinaryTree class manages the binary tree using linked nodes.
- The __init__() function initializes the tree with root = None.
- The set_root() function creates the root node.
- The set_left() function inserts the left child of a parent node.
- The set_right() function inserts the right child of a parent node.
- The search() function searches for a node using a queue.
- The preorder() function traverses in the order Root ---> Left---> Right.
- The inorder() function traverses in the order Left ---> Root ---> Right.
- The postorder() function traverses in the order Left ---> Right ---> Root.
- The level_order() function visits nodes level by level using a queue.
- The display() function displays the tree using preorder traversal.
- The main program creates a binary tree with nodes A, B, C, D, E, F and performs all four traversals.
- The program demonstrates binary tree creation, insertion, searching, and traversal using a linked list.

ERROR TABLE:

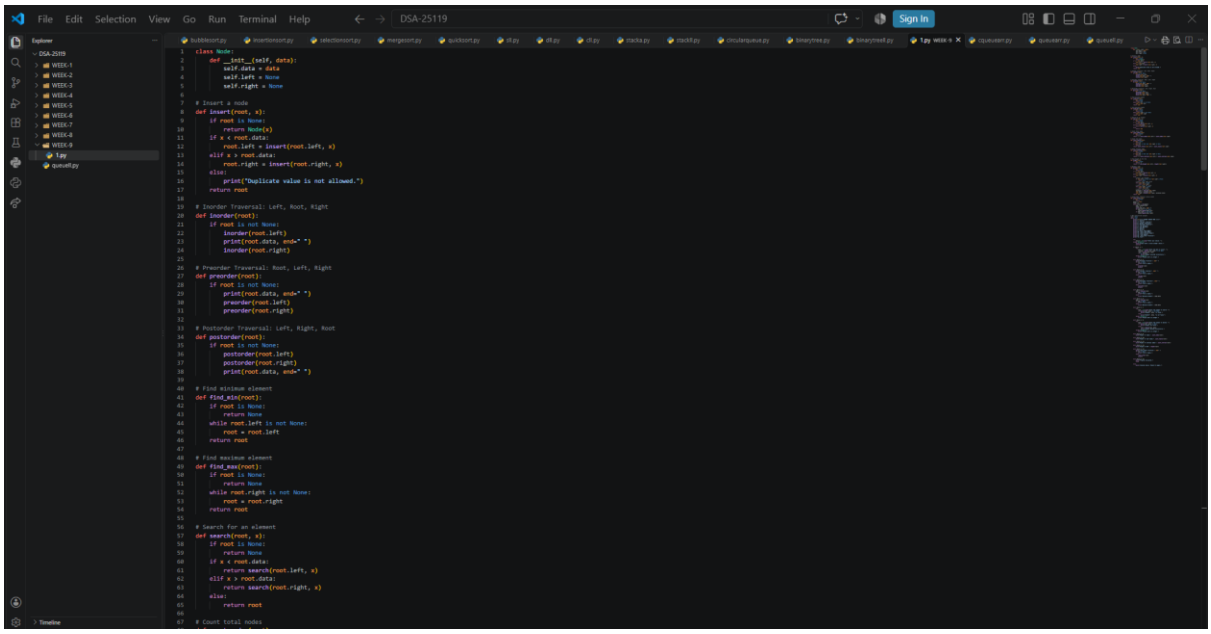
S.no.	Errors	Rectification
1.	**AttributeError** occurred when accessing node links incorrectly.	Used node.left and node.right correctly.
2.	Root already exists error occurred when trying to insert another root.	Checked whether self.root is None before insertion.
3.	Child already exists error occurred when inserting a child into an occupied position.	Checked whether node.left or node.right is None before insertion.

4.	Incorrect traversal output occurred when traversal order was incorrect.	Followed the correct preorder, inorder, postorder, and level-order sequences.

Week-9

AIM: Write a python program to implement a binary search tree with its operations traversals

CODE:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Insert a node
def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root

# Inorder Traversal: Left, Root, Right
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

# Preorder Traversal: Root, Left, Right
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

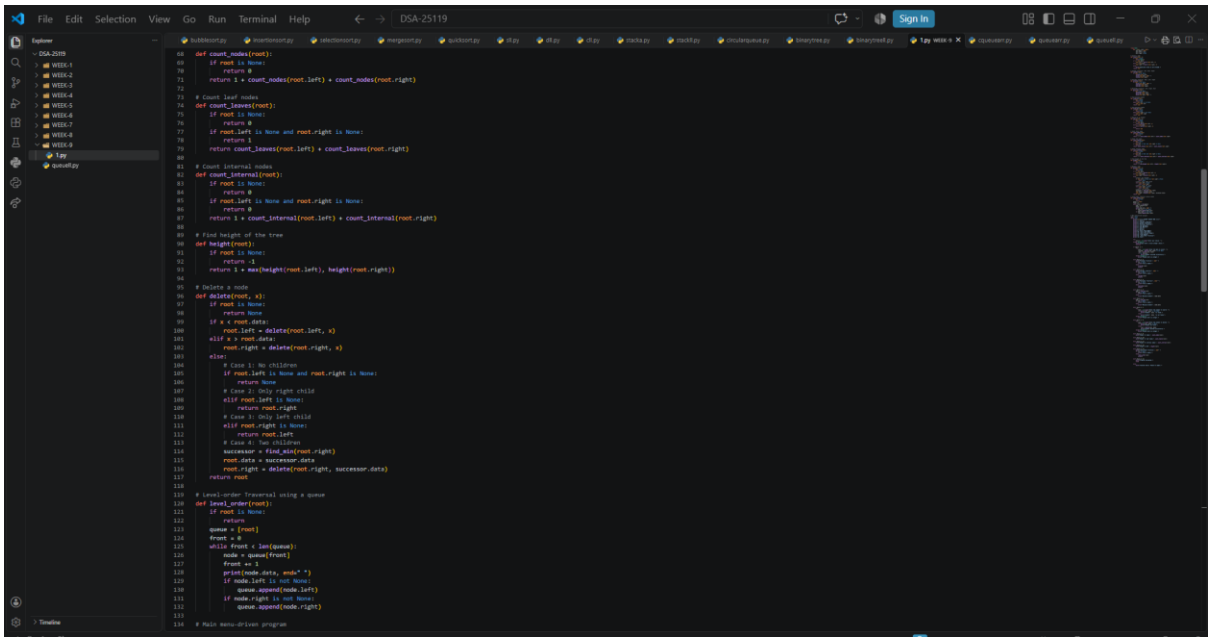
# Postorder Traversal: Left, Right, Root
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Find minimum element
def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

# Find maximum element
def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

# Search for an element
def search(root, x):
    if root is None:
        return None
    if x < root.data:
        return search(root.left, x)
    elif x > root.data:
        return search(root.right, x)
    else:
        return root

# Count total nodes
```



```
def count_nodes(root):
    if root is None:
        return 0
    return 1 + count_nodes(root.left) + count_nodes(root.right)

# Count leaf nodes
def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

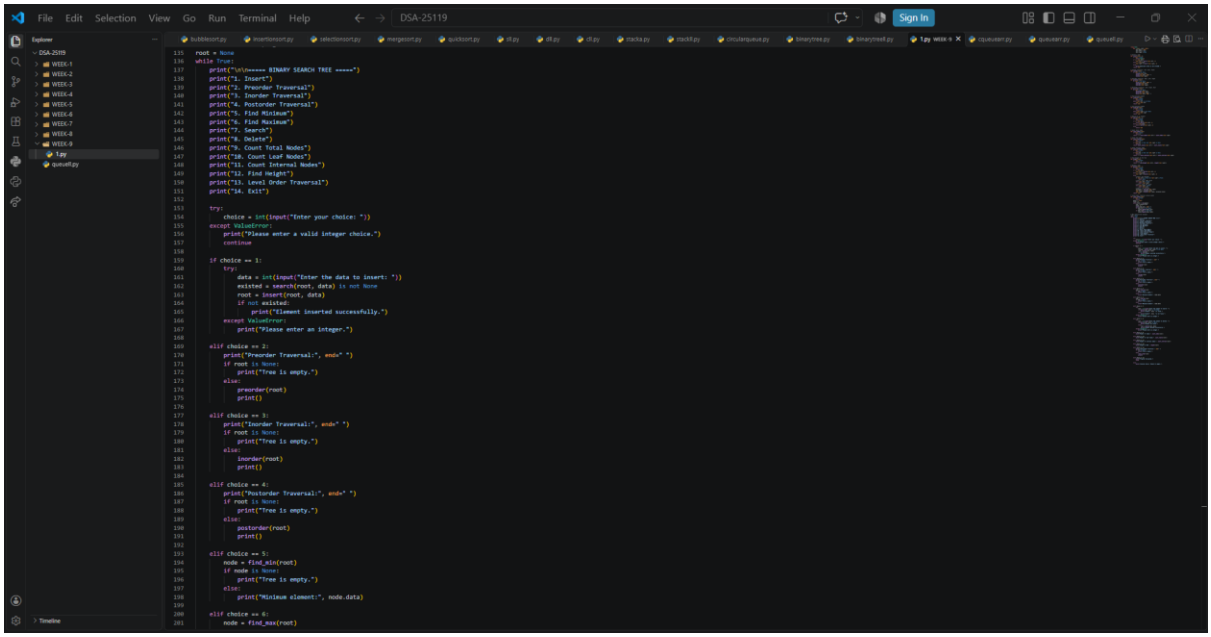
# Count internal nodes
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return 1 + count_internal(root.left) + count_internal(root.right)

# Find height of the tree
def height(root):
    if root is None:
        return -1
    return 1 + max(height(root.left), height(root.right))

# Delete a node
def delete(root, x):
    if root is None:
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: No children
        if root.left is None and root.right is None:
            return None
        # Case 2: Only right child
        elif root.left is None:
            return root.right
        # Case 3: Only left child
        elif root.right is None:
            return root.left
        # Case 4: Two children
        successor = find_min(root.right)
        root.data = successor.data
        root.right = delete(root.right, successor.data)
    return root

# Level-order Traversal using a queue
def level_order(root):
    if root is None:
        return
    queue = [root]
    front = 0
    while front < len(queue):
        node = queue[front]
        front += 1
        print(node.data, end=" ")
        if node.left is not None:
            queue.append(node.left)
        if node.right is not None:
            queue.append(node.right)

# Main menu-driven program
```



```
root = None
while True:
    print("\n\n===== BINARY SEARCH TREE =====")
    print("1. Insert")
    print("2. Preorder Traversal")
    print("3. Inorder Traversal")
    print("4. Postorder Traversal")
    print("5. Find Minimum")
    print("6. Find Maximum")
    print("7. Search")
    print("8. Delete")
    print("9. Count Total Nodes")
    print("10. Count Leaf Nodes")
    print("11. Count Internal Nodes")
    print("12. Find Height")
    print("13. Level Order Traversal")
    print("14. Exit")

    try:
        choice = int(input("Enter your choice: "))
    except ValueError:
        print("Please enter a valid integer choice.")
        continue

    if choice == 1:
        try:
            data = int(input("Enter the data to insert: "))
            existed = search(root, data) is not None
            root = insert(root, data)
            if not existed:
                print("Element inserted successfully.")
        except ValueError:
            print("Please enter an integer.")

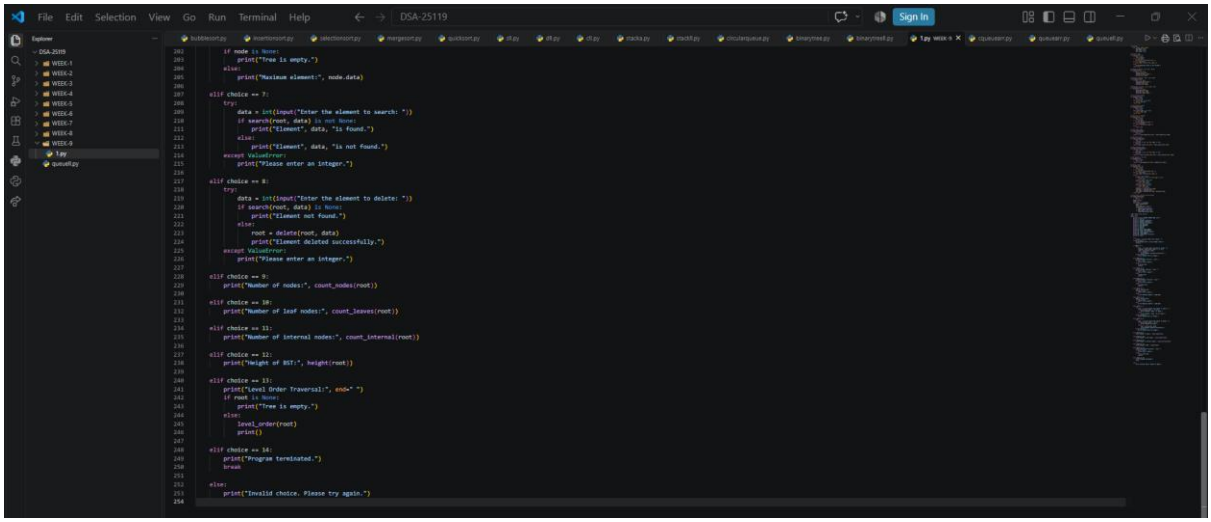
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()

    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()

    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()

    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)

    elif choice == 6:
        node = find_max(root)
```



```
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)

    elif choice == 7:
        try:
            data = int(input("Enter the element to search: "))
            if search(root, data) is not None:
                print("Element", data, "is found.")
            else:
                print("Element", data, "is not found.")
        except ValueError:
            print("Please enter an integer.")

    elif choice == 8:
        try:
            data = int(input("Enter the element to delete: "))
            if search(root, data) is None:
                print("Element not found.")
            else:
                root = delete(root, data)
                print("Element deleted successfully.")
        except ValueError:
            print("Please enter an integer.")

    elif choice == 9:
        print("Number of nodes:", count_nodes(root))

    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))

    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))

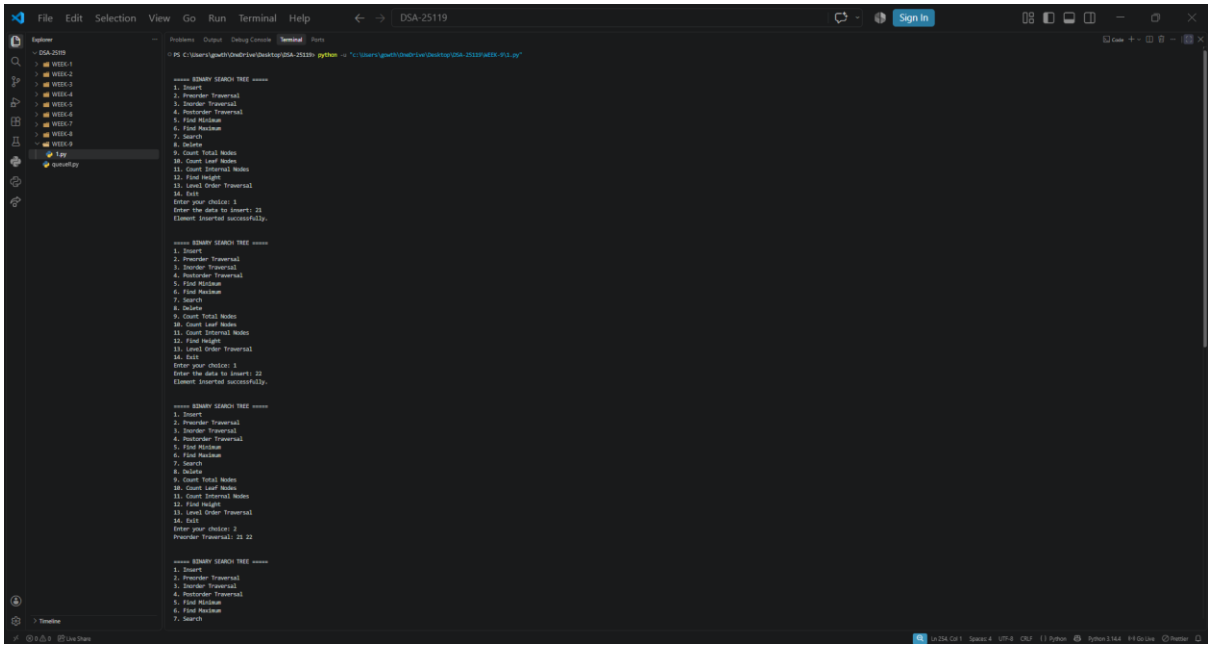
    elif choice == 12:
        print("Height of BST:", height(root))

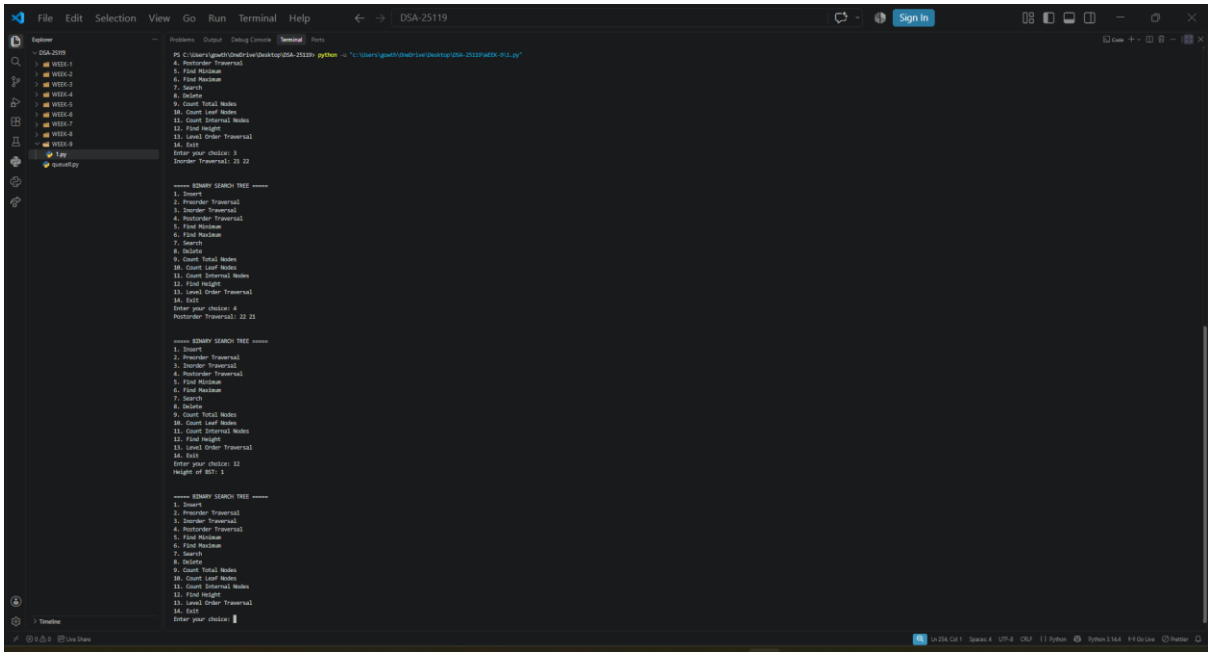
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()

    elif choice == 14:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:





EXPLANATION:

- The Node class creates a node containing data, a left link, and a right link.
- The insert() function inserts values into the BST, placing smaller values on the left and larger values on the right.
- The inorder() function traverses the tree in the order Left → Root → Right.
- The preorder() function traverses the tree in the order Root → Left → Right.
- The postorder() function traverses the tree in the order Left → Right → Root.
- The find_min() function finds the minimum element by moving to the leftmost node.

- The find_max() function finds the maximum element by moving to the rightmost node.The search() function searches for an element using BST comparisons.
- The count_nodes() function counts the total number of nodes. The count_leaves() function counts nodes that have no children.The count_internal() function counts nodes that have at least one child.
- The height() function calculates the height of the tree in edges.
- The delete() function removes a node, handling cases with no children, one child, or two children.The level_order() function visits nodes level by level using a queue.
- The main menu allows the user to select and perform each BST operation, validate inputs, and exit the program.
- The program demonstrates BST insertion, deletion, searching, node counting, height calculation, and four traversal methods.

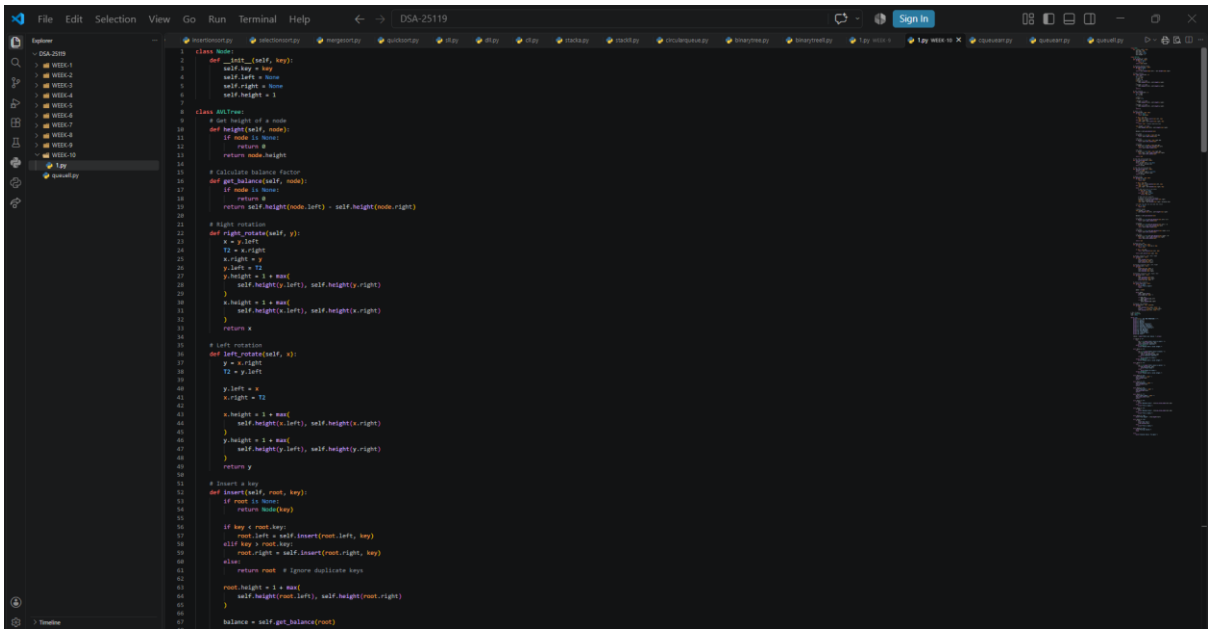
ERROR TABLE:

S.no.	Errors	Rectification
1.	TypeError / ValueError occurred when invalid input was entered.	Used try-except to validate integer inputs.
2.	Duplicate value error occurred when inserting an existing value.	Checked for duplicate values before insertion.
3.	Search error occurred when the element was not present in the BST.	Checked whether search() returns None before displaying the result.
4.	Empty tree error occurred when finding the minimum or maximum of an empty tree.	Checked whether the root is None before performing the operation.
5.	Deletion error occurred when deleting a node that did not exist.	Searched for the element before calling the delete() function.
6.	Incorrect traversal output occurred when the traversal order was incorrect.	Followed the correct preorder, inorder, postorder, and level-order sequences

Week-10

AIM: Write a python program to implement a AVL tree with all its operations.

PROGRAM:



```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    # Get height of a node
    def height(self, node):
        if node is None:
            return 0
        return node.height

    # Calculate balance factor
    def get_balance(self, node):
        if node is None:
            return 0
        return self.height(node.left) - self.height(node.right)

    # Right rotation
    def right_rotate(self, y):
        x = y.left
        T2 = x.right
        x.right = y
        y.left = T2
        y.height = 1 + max(
            self.height(y.left), self.height(y.right)
        )
        x.height = 1 + max(
            self.height(x.left), self.height(x.right)
        )
        return x

    # Left rotation
    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

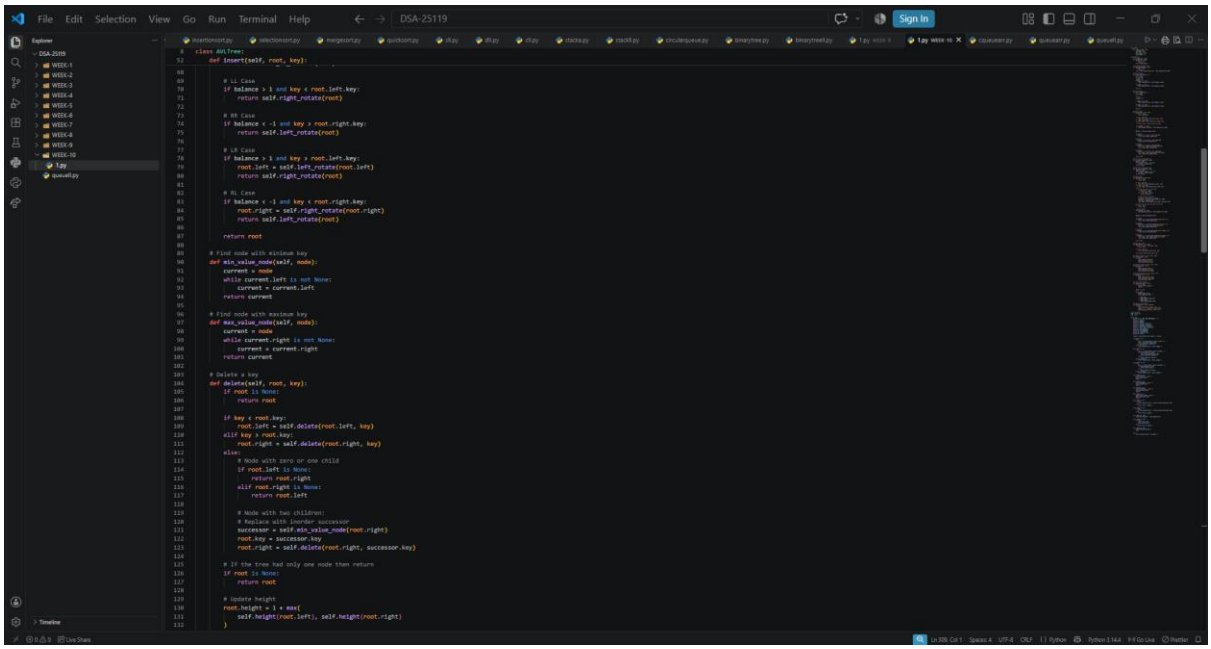
        x.height = 1 + max(
            self.height(x.left), self.height(x.right)
        )
        y.height = 1 + max(
            self.height(y.left), self.height(y.right)
        )
        return y

    # Insert a key
    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root  # Ignore duplicate keys

        root.height = 1 + max(
            self.height(root.left), self.height(root.right)
        )

        balance = self.get_balance(root)
```



```
class AVLTree:
    def insert(self, root, key):

        # LL Case
        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

        # LR Case
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Find node with minimum key
    def min_value_node(self, node):
        current = node
        while current.left is not None:
            current = current.left
        return current

    # Find node with maximum key
    def max_value_node(self, node):
        current = node
        while current.right is not None:
            current = current.right
        return current

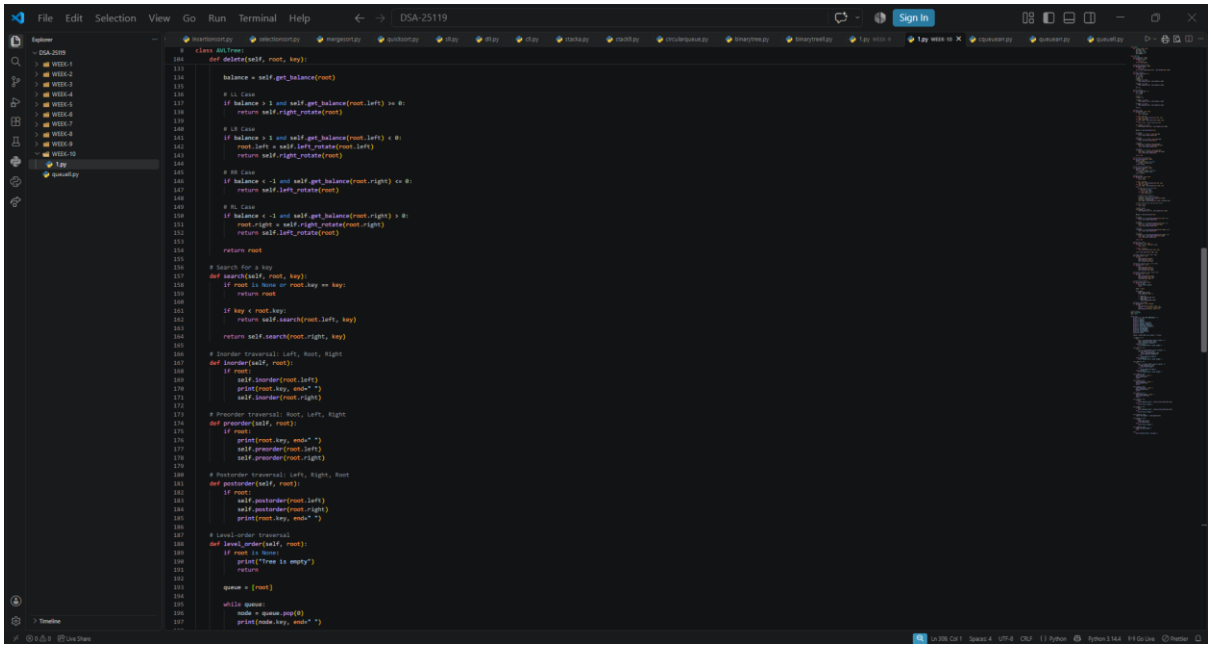
    # Delete a key
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            # Node with zero or one child
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            # Node with two children:
            # Replace with inorder successor
            successor = self.min_value_node(root.right)
            root.key = successor.key
            root.right = self.delete(root.right, successor.key)

        # If the tree had only one node then return
        if root is None:
            return root

        # Update height
        root.height = 1 + max(
            self.height(root.left), self.height(root.right)
        )
```



```
    def delete(self, root, key):

        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        # LR Case
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Search for a key
    def search(self, root, key):
        if root is None or root.key == key:
            return root

        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

    # Inorder traversal: Left, Root, Right
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

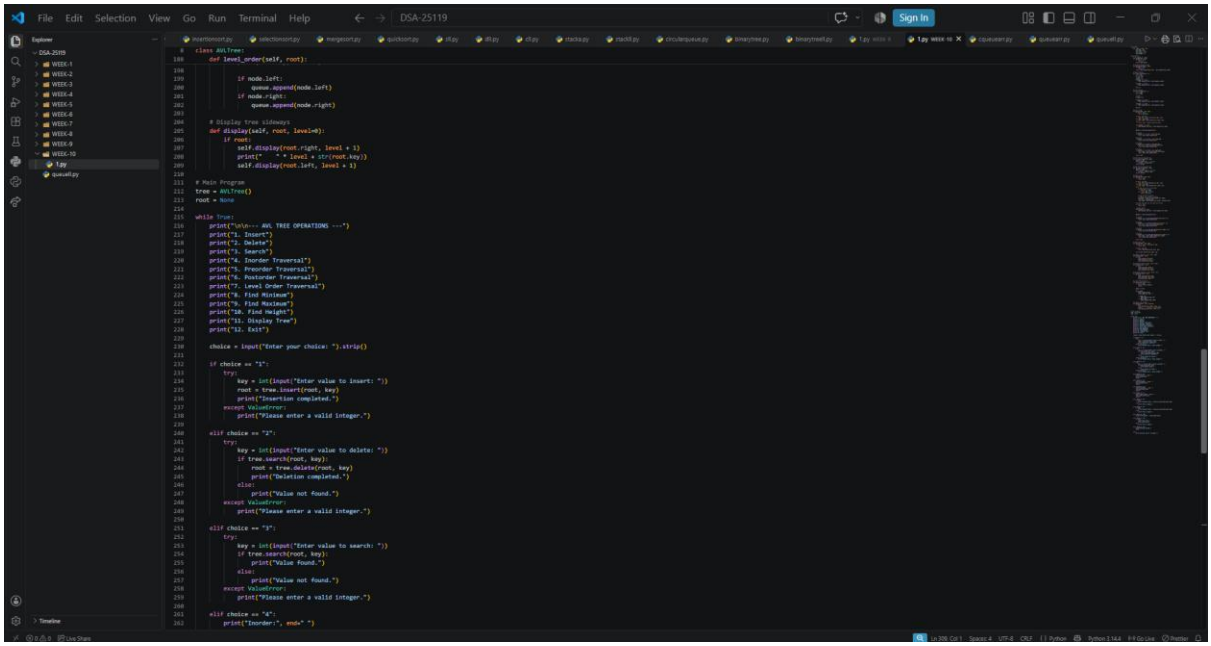
    # Preorder traversal: Root, Left, Right
    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder traversal: Left, Right, Root
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

    # Level-order traversal
    def level_order(self, root):
        if root is None:
            print("Tree is empty")
            return

        queue = [root]

        while queue:
            node = queue.pop(0)
            print(node.key, end=" ")
```



```
    def level_order(self, root):

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

    # Display tree sideways
    def display(self, root, level=0):
        if root:
            self.display(root.right, level + 1)
            print("    " * level + str(root.key))
            self.display(root.left, level + 1)

# Main Program
tree = AVLTree()
root = None

while True:
    print("\n\n--- AVL TREE OPERATIONS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Level Order Traversal")
    print("8. Find Minimum")
    print("9. Find Maximum")
    print("10. Find Height")
    print("11. Display Tree")
    print("12. Exit")

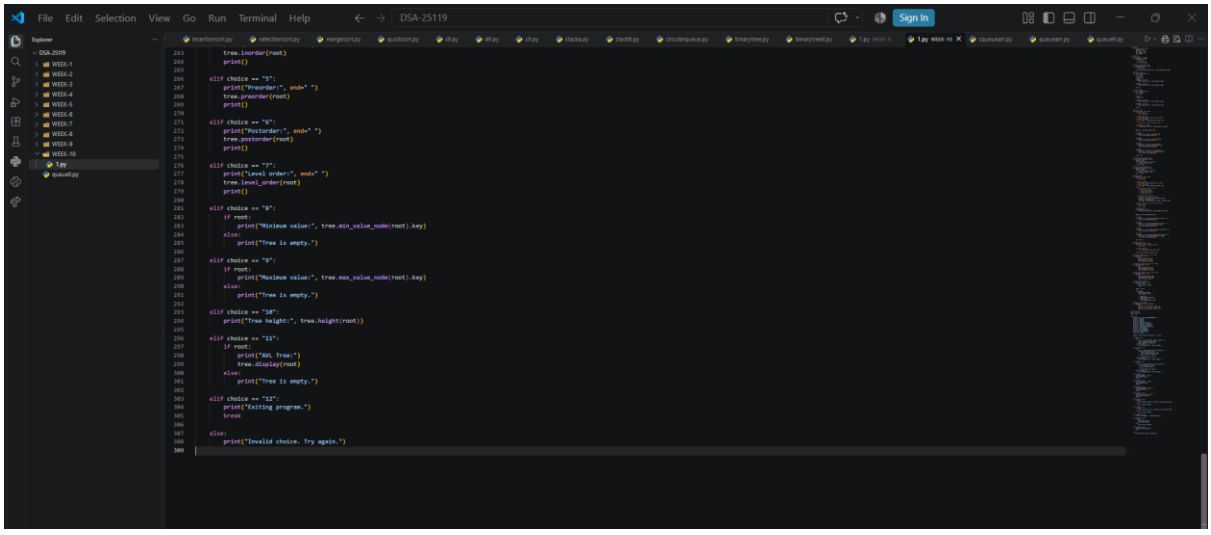
    choice = input("Enter your choice: ").strip()

    if choice == "1":
        try:
            key = int(input("Enter value to insert: "))
            root = tree.insert(root, key)
            print("Insertion completed.")
        except ValueError:
            print("Please enter a valid integer.")

    elif choice == "2":
        try:
            key = int(input("Enter value to delete: "))
            if tree.search(root, key):
                root = tree.delete(root, key)
                print("Deletion completed.")
            else:
                print("Value not found.")
        except ValueError:
            print("Please enter a valid integer.")

    elif choice == "3":
        try:
            key = int(input("Enter value to search: "))
            if tree.search(root, key):
                print("Value found.")
            else:
                print("Value not found.")
        except ValueError:
            print("Please enter a valid integer.")

    elif choice == "4":
        print("Inorder:", end=" ")
```



```
        tree.inorder(root)
        print()

    elif choice == "5":
        print("Preorder:", end=" ")
        tree.preorder(root)
        print()

    elif choice == "6":
        print("Postorder:", end=" ")
        tree.postorder(root)
        print()

    elif choice == "7":
        print("Level order:", end=" ")
        tree.level_order(root)
        print()

    elif choice == "8":
        if root:
            print("Minimum value:", tree.min_value_node(root).key)
        else:
            print("Tree is empty.")

    elif choice == "9":
        if root:
            print("Maximum value:", tree.max_value_node(root).key)
        else:
            print("Tree is empty.")

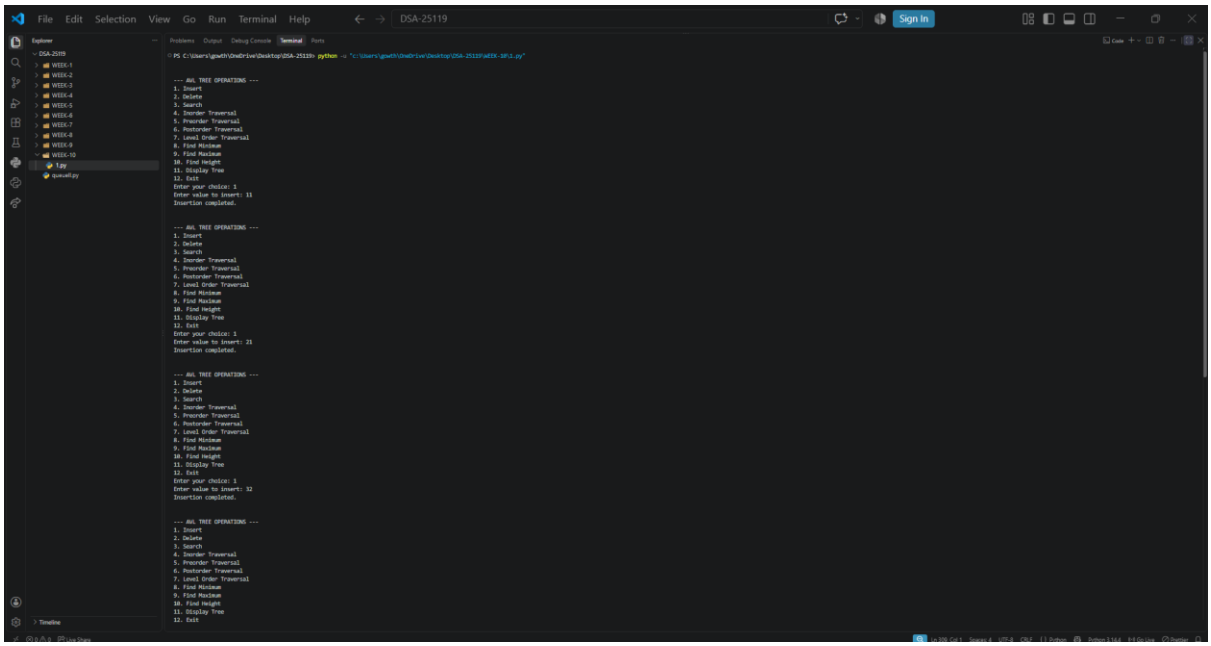
    elif choice == "10":
        print("Tree height:", tree.height(root))

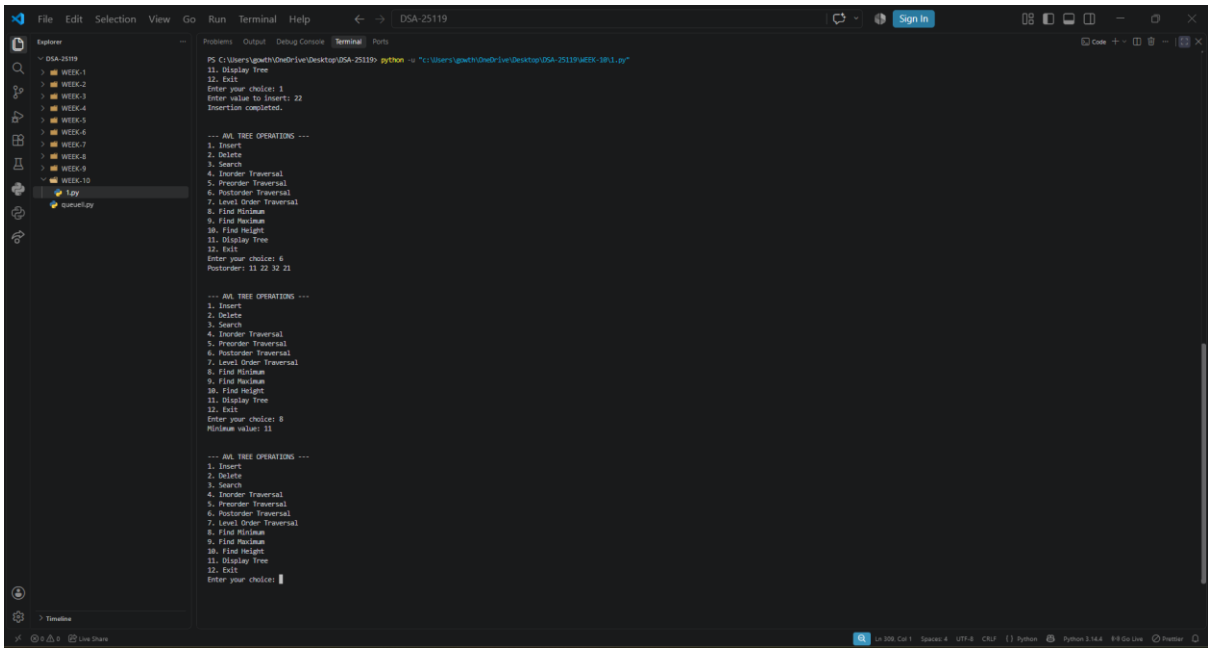
    elif choice == "11":
        if root:
            print("AVL Tree:")
            tree.display(root)
        else:
            print("Tree is empty.")

    elif choice == "12":
        print("Exiting program.")
        break

    else:
        print("Invalid choice. Try again.")
```

OUTPUT:





```
Enter your choice: 11
AVL Tree:
    32
        22
21
    11


--- AVL TREE OPERATIONS ---
```

EXPLANATION:

- The Node class creates a node containing a key, left link, right link, and height.
- The height() function returns the height of a node and returns 0 if the node is empty.

- The get_balance() function calculates the balance factor using the heights of the left and right subtrees.
- The right_rotate() and left_rotate() functions perform rotations to balance the AVL tree.
- The insert() function inserts values into the AVL tree, placing smaller values on the left and larger values on the right.
- The LL, RR, LR, and RL cases handle tree imbalance using appropriate rotations.
- The min_value_node() function finds the minimum element by moving to the leftmost node.
- The max_value_node() function finds the maximum element by moving to the rightmost node.
- The delete() function removes a node, handling cases with no children, one child, or two children, and balances the tree.
- The search() function searches for an element using BST comparisons.
- The inorder() function traverses the tree in the order Left → Root → Right.
- The preorder() function traverses the tree in the order Root → Left → Right.
- The postorder() function traverses the tree in the order Left → Right → Root.
- The level_order() function visits nodes level by level using a queue.
- The display() function displays the AVL tree structure sideways.
- The main menu allows the user to select and perform each AVL tree operation, validate inputs, and exit the program.
- The program demonstrates AVL tree insertion, deletion, searching, balancing, minimum and maximum element finding, height calculation, and traversal methods.

ERROR TABLE:

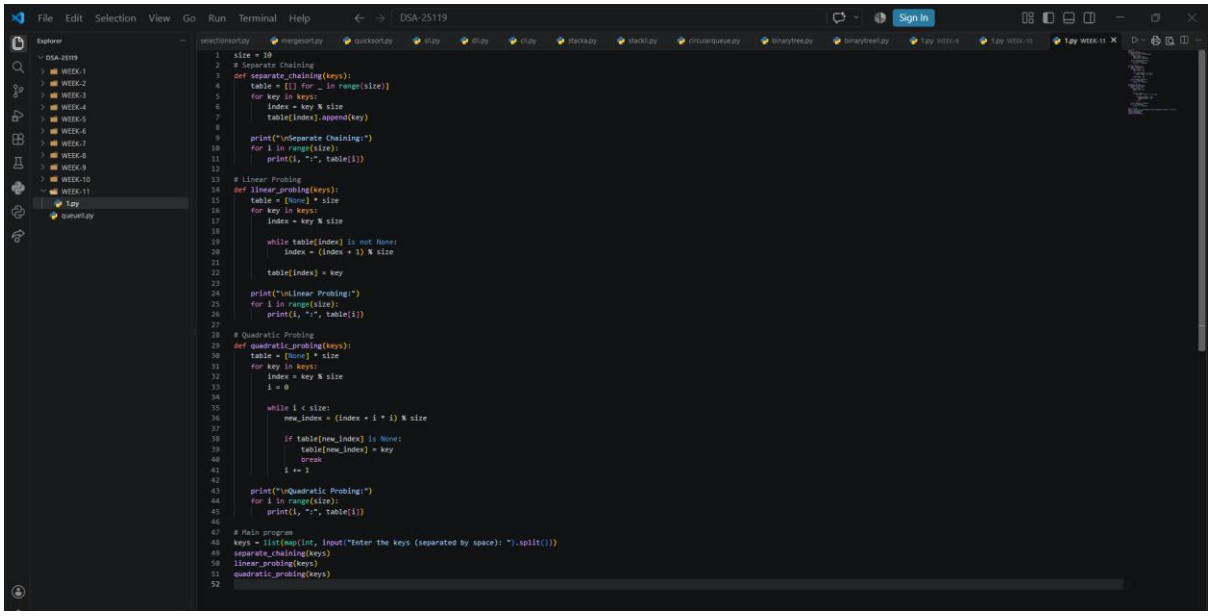
S.no.	Errors	Rectification
1.	ValueError occurred when non-integer input was entered.	Used try-except to validate integer inputs.
2.	Duplicate values were entered during insertion.	Checked for duplicate keys before inserting a new node.
3.	Search failed when the given element was not present in the tree.	Checked whether search() returns None before displaying the result.

4.	Minimum or maximum value could not be found when the tree was empty.	Checked whether the root is None before performing the operation.
5.	Deletion failed when the given element was not present in the tree.	Searched for the element before calling the delete() function.
6.	Tree became unbalanced after insertion or deletion.	Applied appropriate LL, RR, LR and RL rotations.
7.	Traversal operations produced no elements when the tree was empty.	Checked whether the root is None before performing traversal.
8.	Incorrect height was obtained after tree modifications.	Updated node heights after insertion, deletion and rotations.

Week-11

AIM: Write a python program to implement Hashing (Seperate chaining, Linear probing, and Quadratic probing).

PROGRAM:



```
size = 10
# Separate Chaining
def separate_chaining(keys):
    table = [[] for _ in range(size)]
    for key in keys:
        index = key % size
        table[index].append(key)

    print("\nSeparate Chaining:")
    for i in range(size):
        print(i, ":", table[i])

# Linear Probing
def linear_probing(keys):
    table = [None] * size
    for key in keys:
        index = key % size

        while table[index] is not None:
            index = (index + 1) % size

        table[index] = key

    print("\nLinear Probing:")
    for i in range(size):
        print(i, ":", table[i])

# Quadratic Probing
def quadratic_probing(keys):
    table = [None] * size
    for key in keys:
        index = key % size
        i = 0

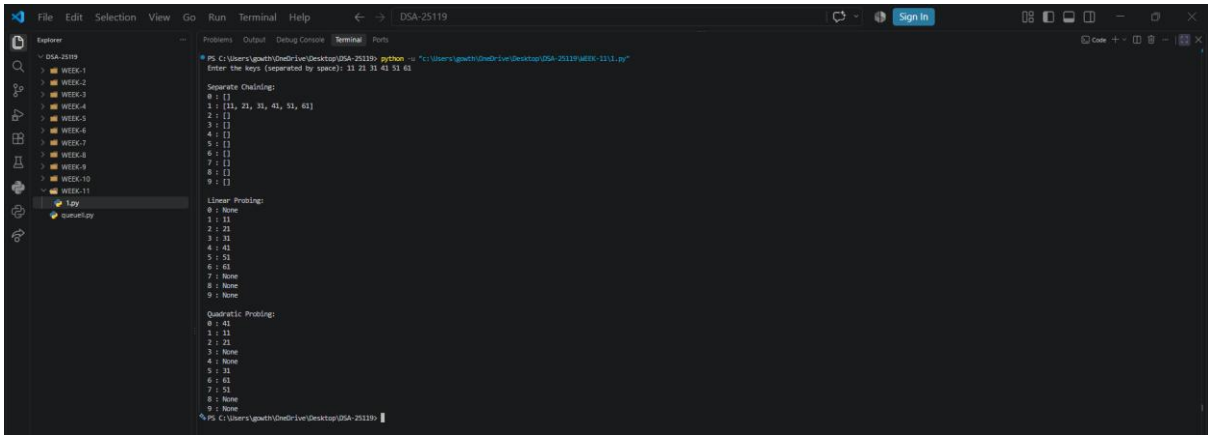
        while i < size:
            new_index = (index + i * i) % size

            if table[new_index] is None:
                table[new_index] = key
                break
            i += 1

    print("\nQuadratic Probing:")
    for i in range(size):
        print(i, ":", table[i])

# Main program
keys = list(map(int, input("Enter the keys (separated by space): ").split()))
separate_chaining(keys)
linear_probing(keys)
quadratic_probing(keys)
```

OUTPUT:



```
Enter the keys (separated by space): 11 21 31 41 51 61

Separate Chaining:
0 : []
1 : [11, 21, 31, 41, 51, 61]
2 : []
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []

Linear Probing:
0 : None
1 : 11
2 : 21
3 : 31
4 : 41
5 : 51
6 : 61
7 : None
8 : None
9 : None

Quadratic Probing:
0 : 41
1 : 11
2 : 21
3 : None
4 : None
5 : 31
6 : 61
7 : 51
8 : None
9 : None
```

EXPLANATION:

- The size variable defines the size of the hash table as 10.
- The separate_chaining() function creates a hash table with empty lists to store the elements.

- The hash function key % size calculates the index for each key. The append() function inserts keys into the corresponding list. If multiple keys have the same index, they are stored in the same list.
- The linear_probing() function creates an empty hash table using None to represent empty positions.
- The while loop checks whether the calculated index is occupied. If a collision occurs, (index + 1) % size searches for the next available position and inserts the key.
- The quadratic_probing() function resolves collisions by checking alternative positions using square increments.
- The expression (index + i * i) % size calculates the new index. The if condition checks whether the position is empty, and the break statement stops the loop after insertion.
- The i += 1 statement increases the counter to check the next position if a collision occurs.
- The main program accepts multiple integer keys from the user using input (), split (), map (), and list ().
- The three functions are called to perform Separate Chaining, Linear Probing, and Quadratic Probing.
- The program demonstrates hashing, collision detection, and collision resolution using three different techniques.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Collision occurred when multiple keys generated the same index.	Applied separate chaining and probing techniques.
2.	Index error occurred when the table reached its last position.	Used modulo operation to wrap around the table.
3.	Insertion failed when the hash table was full.	Checked for available positions before insertion.
4.	Elements were stored at incorrect positions during linear probing.	Used (index + 1) % size to find the next position.

5.	Elements were stored at incorrect positions during linear probing.	Used (index + 1) % size to find the next position.
6.	Incorrect positions were checked during quadratic probing.	Used the formula (index + i * i) % size.